Experiment 10 I²C Communication

ART
10/28/26

Purpose

Use the PIC16F883 MSSP peripheral for I²C communication. Begin with an address only master transaction & measurements of the two-wire bus. Later parts add an external sensor, a second PIC operating as a slave, & UART integration.
Repository Setup: Create one Git repository for Lab 10. Add the .gitignore before staging project files. Keep complete mplab X projects & follow the repo rules.

Standards & references

- RCET 3375 Lab Standard
- RCET PIC-As style guide
- PIC datasheet & Family manual
- AN735: Using PICmicro MSSP module for Master & Slave Communications (I²C)
- TB3252: Using the MSSP in Master mode
- AN736: An I²C Network Protocol for Environmental Monitoring
- I²C-bus specification (NXP UM10204)
- Lab 8 - UART
- Texas Instruments ADS1113 data sheet (SBAS444E)

Equipment & Materials

Part 1: One PIC16F883 Minimum circuit with a 4MHz crystal, 5V supply, PICkit3, MPLabX/PIC-as, ~~normal~~ single I²C pull-up resistor in common values near the four calculated test targets (two of each chosen values, one per bus line), Breadboard components, multimeter, O-scope with I²C decoding.
Later Parts: A second PIC16F883 circuit, supplied I²C sensor breakout & input source, Lab 8 UART host application & serial adapter, ADC sensor & servo components, & optional Mastery hardware.

~~Part 1 - I²C Master & bus Timing~~

Part 1 - I²C Master & Bus Timing

Goal

Use one PIC16F883 as a 100kHz I²C Master. Send a periodic address-Only write transaction with no slave connected. Detect the expected NACK, generate Stop, & measure the bus waveforms.

Required Behavior

- Use a 4MHz crystal & a 5V PIC16F883 Circuit. Connect RC3/SCL (Pin 14) & RC4/SDA (Pin 15) to 5V through separate 4.7kΩ pullup resistors. Leave the I²C bus without a slave for this part. Configure the pins for MSSP open-drain operation.
- Configure the MSSP for 7-bit I²C master mode at 100kHz. Address 0x20 with R/W=0. The address & R/W bit form the transmitted byte 0x40 (0100 0000 binary).
- For every probe, generate START, send the address/R-W byte, read the Ack/NACK result, & generate STOP. With no slave, the ninth clock pulse must show a NACK (SDA High).
- Use a hardware timer to start repeated address transmissions. Choose a timer interval that leaves a clear idle gap between the stop of one transaction & the start of the next. Calculate this interval.
- Use RA0 as a NACK indicator. Clear RA0 at the beginning of each probe; make it High if the slave doesn't acknowledge. Leave it Low for Ack. Complete the transaction with stop either way.
- (Continued on next page)

Required Behavior Cont.

- Use RA1 as a timer diagnostic. Set RA1 High when the timer event starts a new transaction, just before calling AddressProbe. Set it Low after AddressProbe returns, following Stop. Each RA1 rising edge marks the start of a new transaction & provides an O-Scope trigger without I²C decoding.

Before Lab

Prepare the following:

- A schematic with the PIC power, reset, oscillator, Programming connections, Separate SDA/Scl pullups, & RA0/RA1 diagnostics. Account for every used pin & components.
- PIC16F883 SFR documentation for MSSP master setup, Start/Stop control, address transmission, Completion, & Ack status, the selected timer's setup & event indication, & I/O configuration. Include the relevent register maps & the values you plan to use. (Pg 294 298)
- Flowcharts for the mainloop & AddressProbe subroutine. Show the timer event, RA1 signal, Start, address transmission, NACK detection, Stop, & return.
- Predicted SCL clock period & fraquency & the interval between transactions based on timer.
- Pull-up resistor investigation: Starting with the nominal 4.7kΩ resistor, calculate the target resistance for nominal, nominal X10, nominal/2, & nominal/10. For each target, select & document a nearby common resistor value that is actually available. Use one resistor per bus line for each test; do not combine resistors. Record the calculated target, chosen standard value, resistor rating, & predicted electrical results for each test.
- For Both SDA & SCL at every selected resistance, predict the idle High voltage, current through pull-up with the bus idle & with the line driven Low, & 30% – 70% rise time. State the assumed bus capacitance & Low voltage. Compare predictions with the I²C standard-mode electrical requirements & PIC16F883 pin limits before applying power.
- Compile the partl source & program the PIC before starting In The Lab steps.

MSSP timing example: For a 4MHz Oscillator & 100kHz I²C clock, find the master baud-rate value using the data sheet equation.

$$F_{SCL} = \frac{F_{OSC}}{4(SSPADD+1)} \qquad SSPADD = \left(\frac{F_{OSC}}{4 F_{SCL}}\right) - 1 = \left(\frac{4MHz}{4 \cdot (100kHz)}\right) - 1 = 9 \qquad T_{SCL} = \frac{1}{100kHz} = 10\mu s$$

Pull-up voltage & current example: With neither SDA nor SCL driven Low, each line should be close to the 5V supply & the ideal pull up current is zero. For a low line, estimate the current through one nominal 4.7kΩ resistor using an assumed 0.4V Low level.

$$V_{idle} \approx V_{DD} = 5V \qquad I_{idle} = \frac{V_{DD} - V_{idle}}{R_P} = 0mA \qquad I_{Low} = \frac{V_{DD} - V_{OL}}{R_P} \approx \frac{5V - .4V}{4.7k\Omega} = .979mA$$

I²C rise-time calculation: Begin with the RC charging equation used in class. Let V_i be the initial line voltage, V_f the final voltage, & $\tau = R_P C_{bus}$ time constant.

$$V(t) = V_f - (V_f - V_i)e^{-t/\tau} \qquad \tau = R_P C_{bus}$$

For the pull-up estimate, assume the line starts at $V_i = 0V$ & rises toward $V_f = V_{DD}$. Solve the charging equation for the time at which the line reaches a chosen fraction of V_{DD}.

$$V(t) = V_{DD}(1 - e^{-t/\tau}) \qquad \frac{V(t)}{V_{DD}} = 1 - e^{-t/\tau} \qquad e^{-t/\tau} \approx 1 - \frac{V(t)}{V_{DD}} \qquad t = -\tau \ln\left(1 - \frac{V(t)}{V_{DD}}\right)$$

ART 10/28/26

Experiment 10 I^2C Communication

AHT 10/28/26

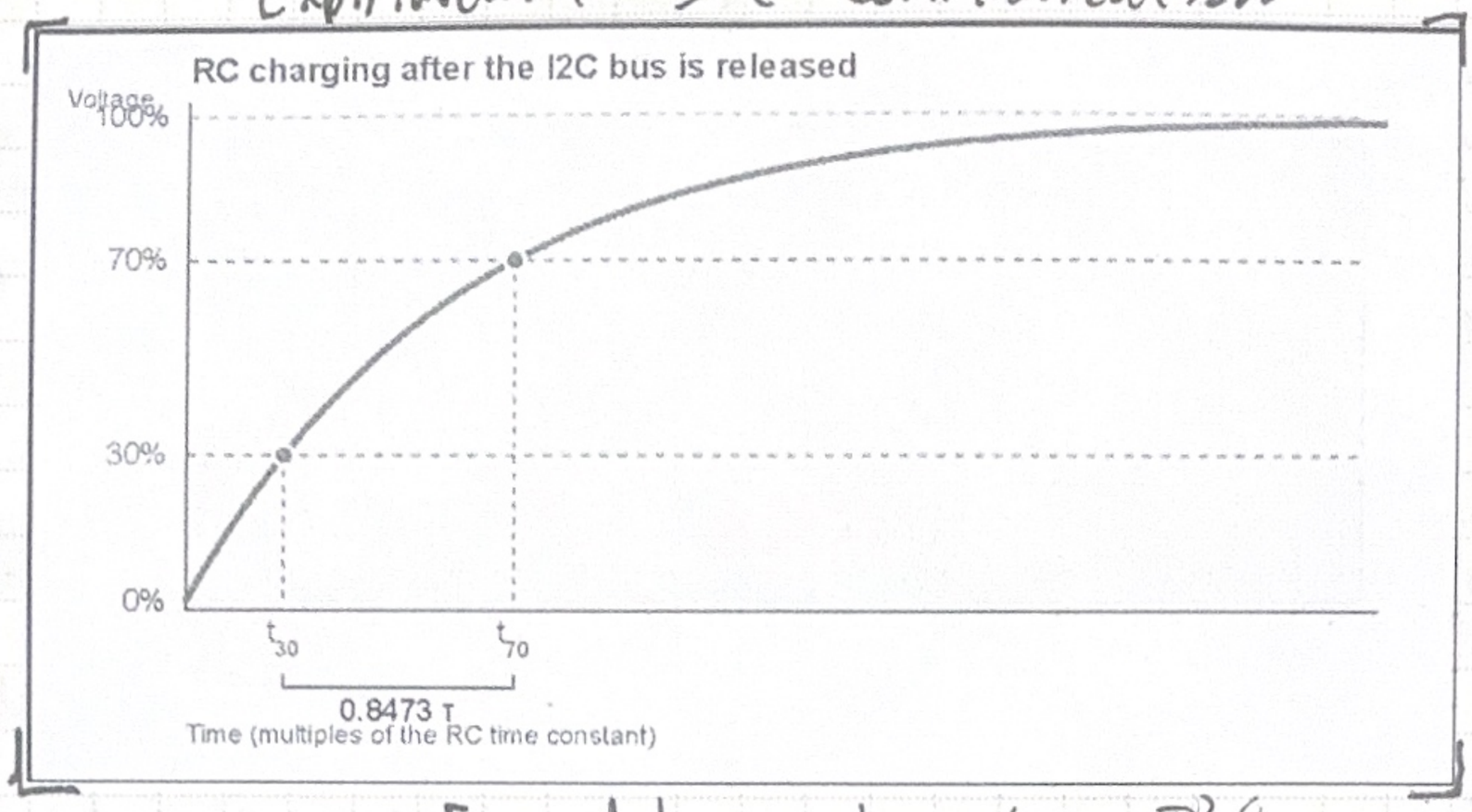


Figure 1-1 cap charge time I^2C

I^2C rise time is measured from 30% to 70% of the bus voltage. Calculate the time to reach each threshold, then subtract:

$t_{30\%} = -\tau \ln(1-0.30) \approx .35625\,\tau$ $\quad T_{70\%} = -\tau \ln(1-.70) \approx 1.203973\,\tau$

$t_r = t_{70\%} - t_{30\%} \approx 0.8473\,\tau = 0.8473\, R_p C_{bus}$

Worked Example: Assume a nominal 4.7kΩ pull-up & 100pF bus capacitance. The capacitance is an example assumption, not a measured value.

$\tau = R_p C_{bus} = 4.7k\Omega \cdot 100pF = 470ns$ $\quad t_r = 0.8473\,(470ns) \approx 398.23ns$

Standard-mode I^2C specifies a maximum bus rise time, & its low-level current spec is based on 3mA. The lowest-resistance test can require substantially more low-state current. Check whether each chosen resistor meets the PIC pin current/voltage limits & I^2C electrical requirements; explain any out of specification prediction. Do not power a circuit that exceeds spec. The circuit has capacitance regardless of a slave being connected.

Required Setup

```
MSSP:
    Set up I2C master operation at 100 kHz
    Enable the peripheral for SDA and SCL

Timer:
    Configure a hardware timer for a repeat interval that separates address transactions
    Clear its event indication and start it

I/O:
    Set RA0 as a digital output for the NACK indicator
    Set RA1 as a digital output for the timer diagnostic
    Configure the other used pins for their required functions

Startup:
    Initialize both diagnostic outputs LOW
```

*Used Timer0 loaded with 0x132 for a timer period of 20ms

AHT 10/28/26

Experiment 10 I²C Communication

ART 10/28/26

Program Structure

```
Main:
    Wait for the chosen timer's event
    Clear the timer event indication
    Set RA1 HIGH
    Call AddressProbe
    Set RA1 LOW
    Repeat

AddressProbe:
    Clear the NACK indicator
    Generate a START condition and wait until it completes
    Send the slave address with the write bit and wait until the byte is sent
    Check the acknowledge result
    If the result is NACK, turn on the NACK indicator
    Generate a STOP condition and wait until it completes
    Return
```

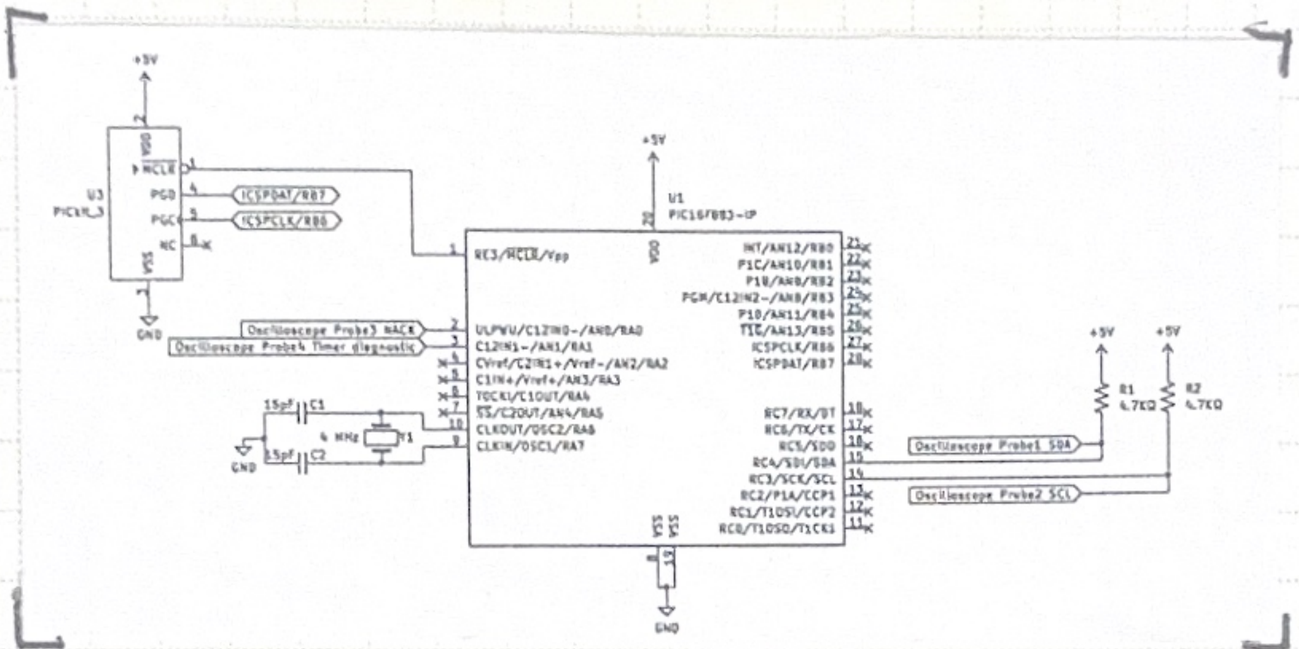
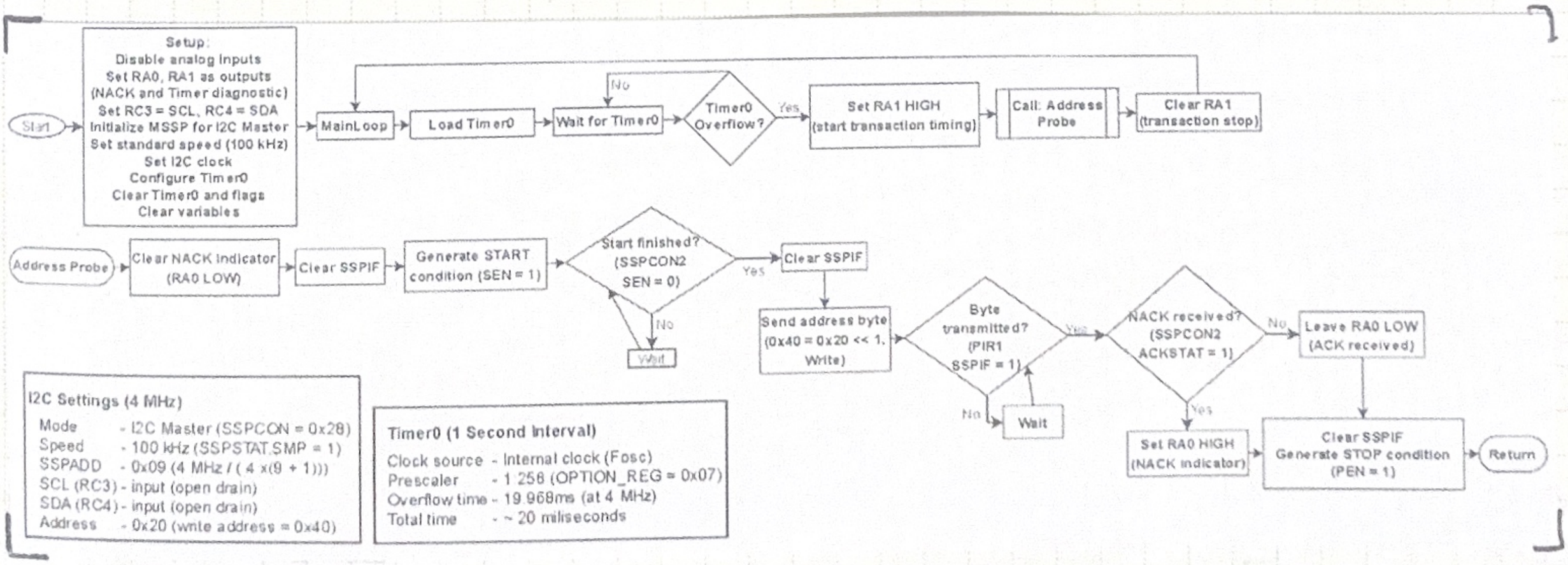


Figure 1-2 I²C diagnostic schematic

Calculated Target	*Standard Value*	*Resistor Rating*	*Predicted Electrical Results*	*Idle HIGH*	*Idle Current*	*LOW Current*	*Rise Time*
4.7 kΩ	4.7 kΩ	500mW	398.09 ns	5V	0mA	0.979 mA	0.398 µs
47 kΩ	47 kΩ	500mW	3.981 us	5V	0mA	0.098 mA	3.982 µs
2.35 kΩ	2.2 kΩ	500mW	186.34 ns	5V	0mA	2.091 mA	0.186 µs
470 Ω	470 Ω	500mW	39.809 ns	5V	0mA	9.787 mA	0.040 µs

The PIC16F883 datasheet spec for max output current for sink/source is 25mA, making each of the resistors in spec. However, the 47 kΩ resistor has the problem that it draws less current and produces a much slower rising edge, which exceeds the Standard-mode timing limit under the 100 pF assumption. The 470 Ω resistor finds the problem of power, which can be less effective when using things like batteries.

Table 1-1 predicted resistor Values, Currents, & rise times



FC 1-1 for code 1-1

Exptriment 10 I^2C Communication

ART 10/30/26

FCH for Code H Constant I^2C Master Addressing.

```
Main:
    ; Start a new timer interval
    BANKSEL TMR0
    MOVLW 0xB2
    MOVWF TMR0
    BCF INTCON,2

TimerWait:
    ; Wait for a Timer0 overflow
    BTFSS INTCON,2
    goto $-1
    BCF INTCON,2          ; Clear overflow flag

    ; Timer event starts new I2C transaction
    BANKSEL PORTA
    BSF PORTA,1           ; RA1 HIGH
    CALL AddressProbe
    BANKSEL PORTA
    BCF PORTA,1           ; RA1 LOW after STOP
    goto Main


AddressProbe:
    ; Clear NACK indicator before each probe
    BANKSEL PORTA
    BCF PORTA,0           ; RA0 LOW

    ; Generate START condition
    BANKSEL PIR1
    BCF PIR1,3            ; Clear SSPIF
    BANKSEL SSPCON2
    BSF SSPCON2,0         ; SEN = 1, start I2C

    BTFSC SSPCON2,0       ; Wait until START finishes
    goto $-1

    ; Send 7-bit address 0x20 with R/W = 0
    BANKSEL PIR1
    BCF PIR1,3            ; Clear SSPIF
    MOVLW 0x40            ; Address = 0x20, Write = 0
    BANKSEL SSPBUF        ; The address is moved left once making 0x20 -> 0x40
    MOVWF SSPBUF          ; Send address byte

    BANKSEL PIR1
    BTFSS PIR1,3          ; Wait for transmission
    goto $-1

    ; Check ACK or NACK
    BANKSEL SSPCON2
    BTFSS SSPCON2,6       ; ACKSTAT = 1 means NACK
    goto SendStop         ; ACK received
    BANKSEL PORTA
    BSF PORTA,0           ; NACK received, RA0 HIGH

SendStop:
    ; Generate STOP condition
    BANKSEL PIR1
    BCF PIR1,3
    BANKSEL SSPCON2
    BSF SSPCON2,2         ; PEN = 1, generate STOP

    BTFSC SSPCON2,2       ; Wait until STOP finishes
    goto $-1
    RETURN
```

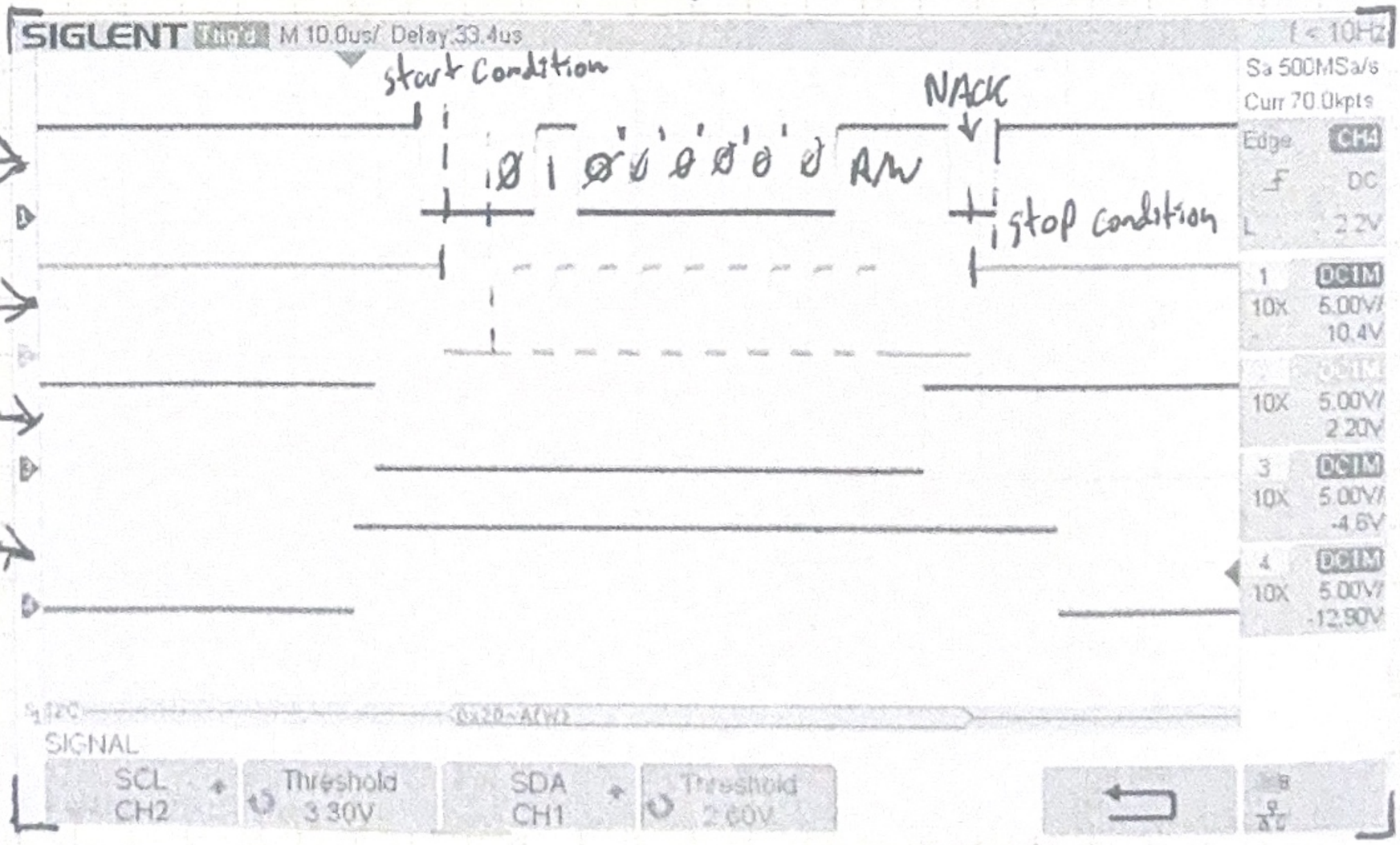


Figure 1-3 Measured Addressing I^2C with Diagnostic Pins

TSL 10-30-26

ART 10/30/26

ART
10/30/26

Part 2 - I²C Master With an ADS1115 Sensor Slave

Goal

Use the Part 1 I²C Master to identify an ADS1115, inspect & change its configuration, & read its ADC results. Observe the conversion-ready signal to choose a suitable reading interval.

Before Lab

Prepare:

- Your working Lab 10 Pt 1 address probe program, Compiled & programmed into the PIC.
- Skim the ADS1115 data sheet. Find the pin descriptions, supply/input limits, & I²C address selection. You will investigate the registers & configure the ADC as you work through the Lab.

Figure 2-1 ADS1115 Block Diagram

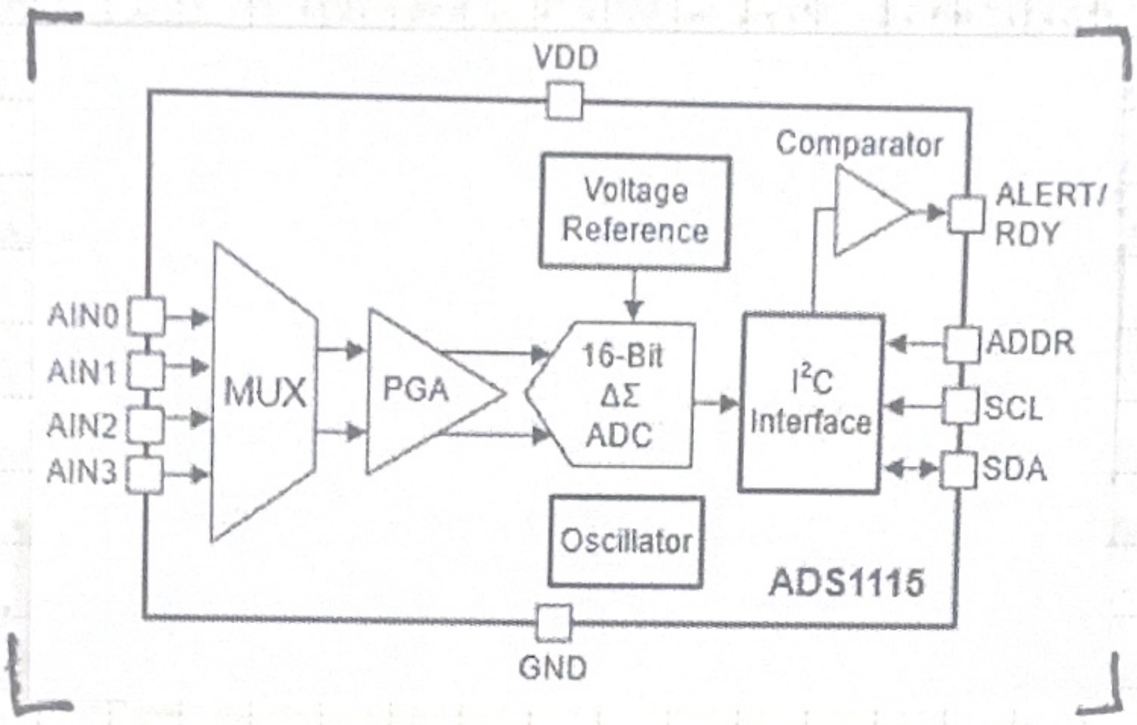
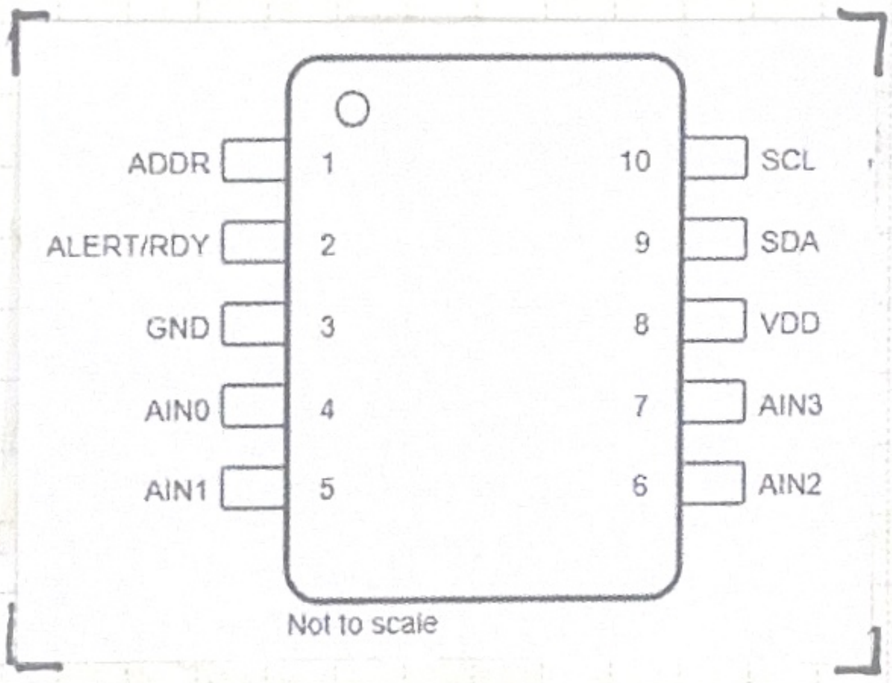


Figure 2-2 ADS1113 Pinout



ADDR PIN CONNECTION	TARGET ADDRESS
GND	1001000b
VDD	1001001b
SDA	1001010b
SCL	1001011b

Table 2-1 Address Selection for ADS1115

		MIN	MAX	UNIT
Power-supply voltage	VDD to GND	–0.3	7	V
Analog input voltage	AIN0, AIN1, AIN2, AIN3	GND – 0.3	VDD + 0.3	V
Digital input voltage	SDA, SCL, ADDR, ALERT/RDY	GND – 0.3	5.5	V
Input current, continuous	Any pin except power supply pins	–10	10	mA
Temperature	Operating ambient, T_A	–40	125	°C
	Junction, T_J	–40	150	
	Storage, T_{stg}	–60	150	

Table 2-2 Absolute max ratings for ADS1115

ART
10/30/26

Experiment 10 I²C Communication

ART 10/30/26

Suggested Implementation (optional)

The three-level I²C software design guide offers one possible implementation with flowcharts, pseudocode, & I²C transaction sequences. It is a reference, not required program structure. You may adapt it or develop your own implementation that meets the Lab requirements; no particular routine layout or number of source files is required.

In the Lab

1. Find the ADS1115 address. Connect the breakout to the Part I SDA/SCL bus. Before Applying power, check the board's supply & I²C logic levels against the PIC circuit. Document the connections in your schematic.

*Pull up hint: With power off & the breakout disconnected from the PIC & external pull-up's, measure resistance with a DMM from SDA & SCL to the Board's supply rail. Check ADDR & ALERT/RDY the same way. These measurements can reveal installed pull-ups; inspect the board markings or schematic to confirm because other onboard components may affect an in-circuit resistance reading. With 4.7k resistors in parallel, these shouldn't be problematic.

Find the four possible ADS1115 addresses in the data sheet. With ADDR tied to GND, use the Part I probe to try all four addresses, recording ACK or NACK for each. Power off before moving ADDR to VDD, Then probe all four addresses again. Modify the probe so each trial runs once after MCLR reset, then enters a do-nothing loop instead of repeating. For each scope capture, arm the O-Scope in Single Sequence mode, then press & release MCLR.

```
For each candidate: START → [address + Write bit] → ACK or NACK → STOP
```

Record all eight results in a table:

7-bit address from data sheet	Transmitted write byte	ADDR = GND	ADDR = VDD
GND 1001000	10010000	(ACK) / NACK	ACK / (NACK)
VDD 1001001	10010010	ACK / (NACK)	(ACK) / NACK
SDA 1001010	10010100	ACK / (NACK)	ACK / (NACK)
SCL 1001011	10010110	ACK / (NACK)	ACK / (NACK)

Table 2-3

Transmitted Address Bytes

Save a decoded ACK capture for each ADDR condition. For the remaining steps, choose either ADDR connection that produced an ACK & use its confirmed address. In the sequences below, ADDR+W & ADDR+R mean the transmitted address byte formed from your chosen 7-bit address with the write bit(0) or read bit(1).

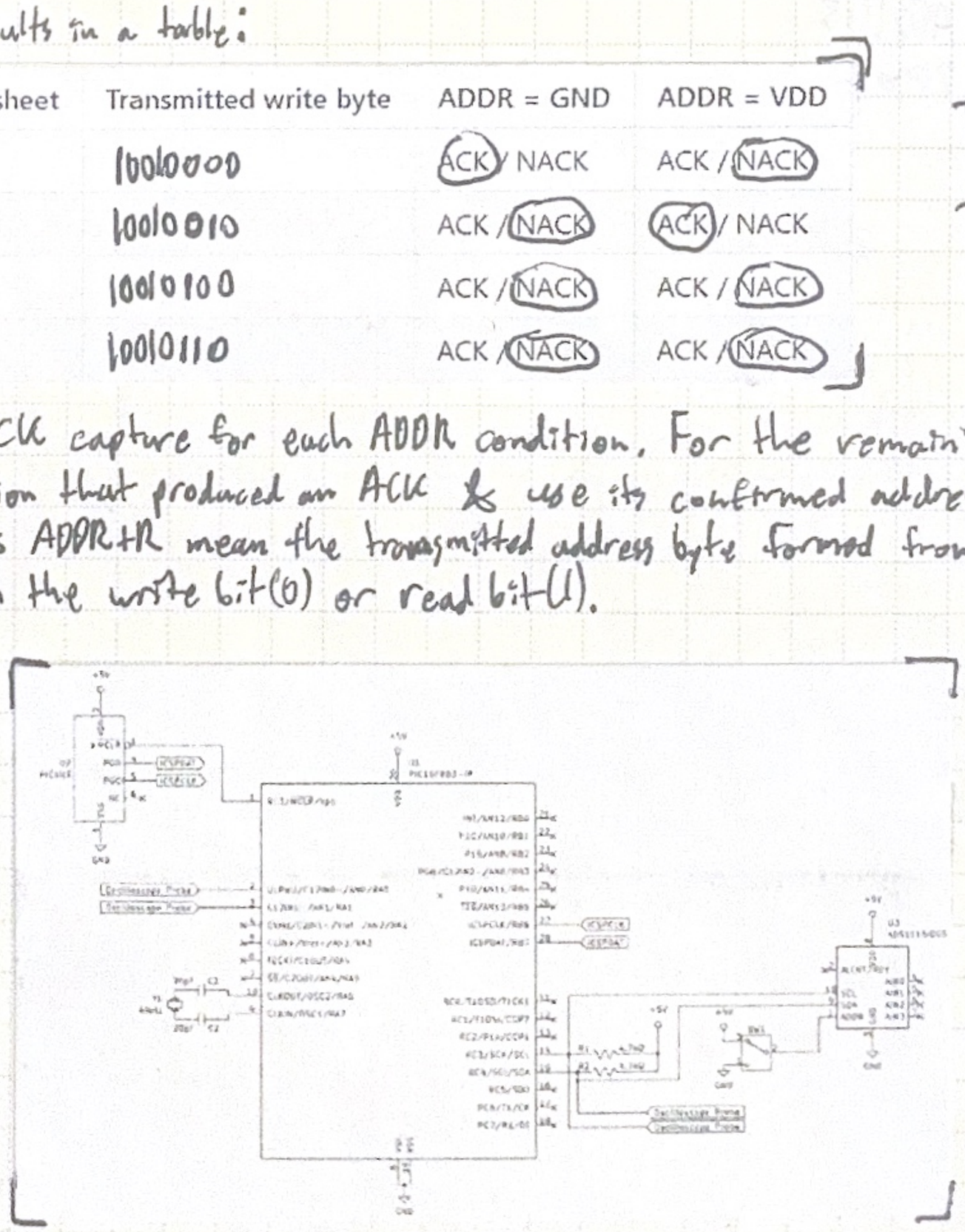
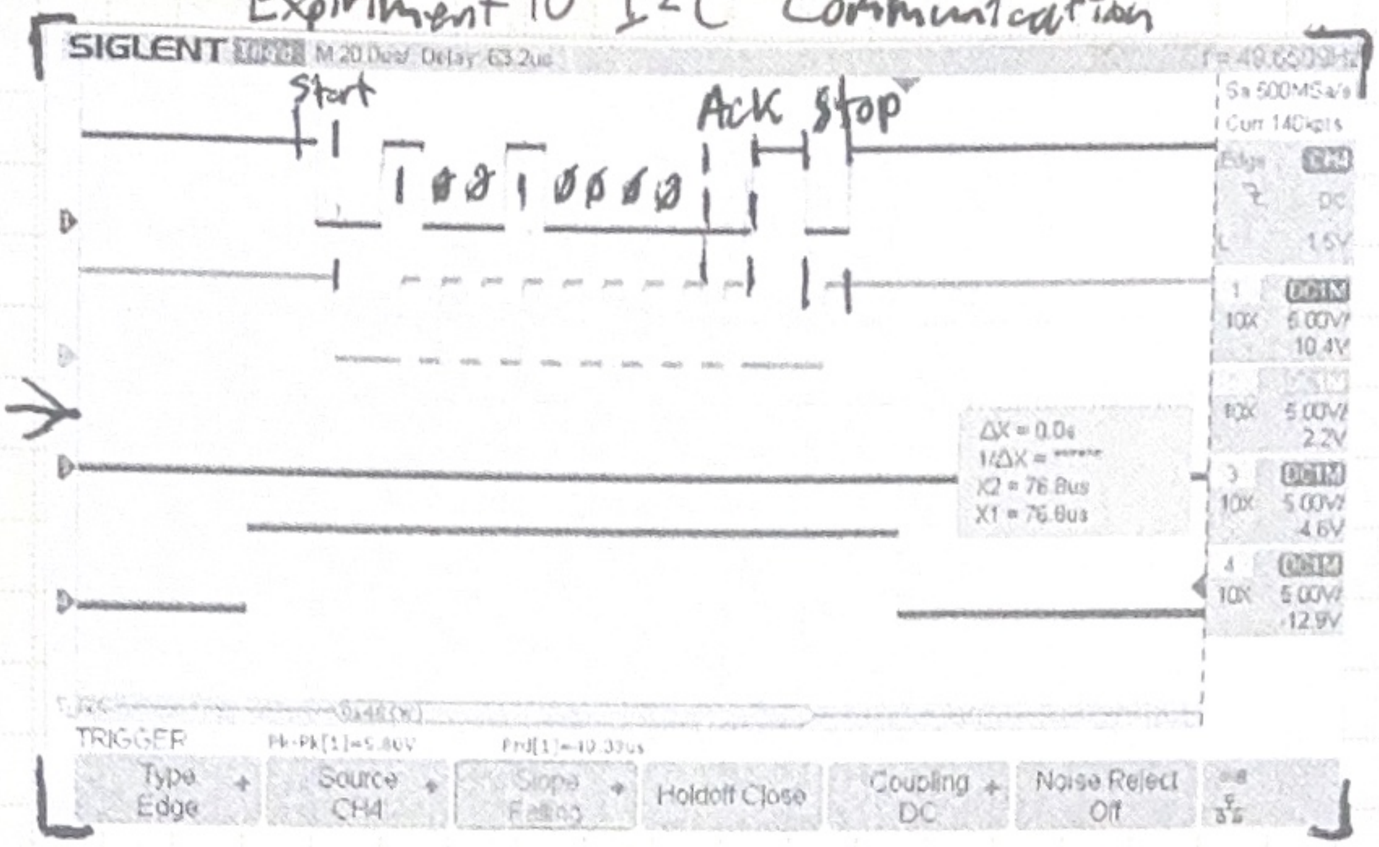


Figure 2-3

Connected PIC with ADS1115

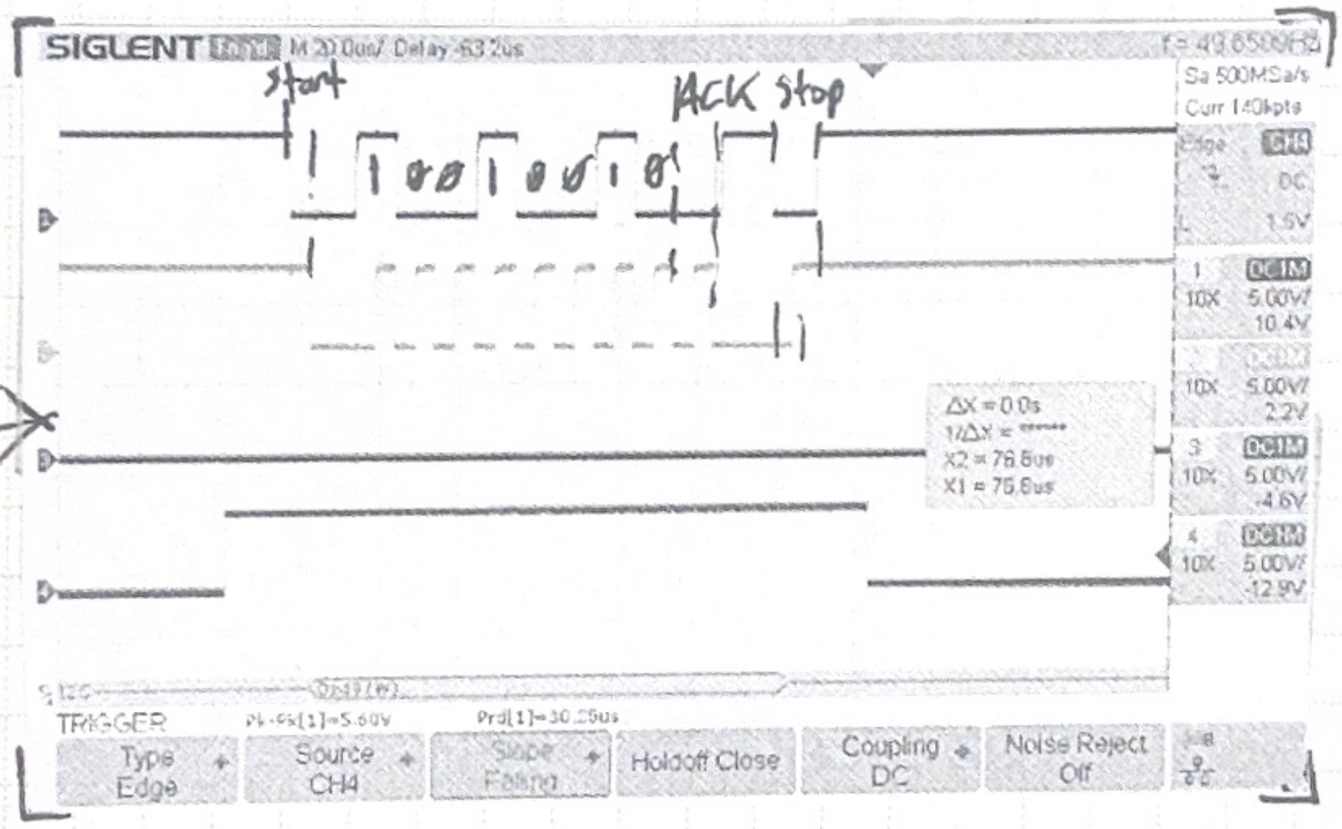
Experiment 10 I²C Communication

ART
11/2/26



Diagnostic shows → no NACK received

Figure 2-4 Address sent = 0x90 ADDR = GND



Diagnostic shows → no NACK received

Figure 2-5 Address sent = 0x92 ADDR = VDD

2. Read the config register. Find the ADS1115 register-pointer table. The one-byte pointer 0x01 selects config; it does not overwrite config. On a freshly powered ADS1115, the expected config value is 0x8583. Select the pointer, then read the 16 bit register with a new START.

```
Select Config (once):
START → ADDR+W → ACK → 0x01 → ACK → STOP

Read Config (once):
START → ADDR+R → ACK → Config MSB → PIC ACK → Config LSB → PIC NACK → STOP
```

Run the pointer write & Config read once after MCLR reset, then enter a do-nothing Loop. Arm the O-Scope in Single Sequence mode before releasing MCLR. Combine the two bytes & explain the observed Config fields using the data sheet. MCLR resets the PIC, not the ADS1115; to check the ADS1115's power-up value, power-cycle the ADC before this test. the ACKs after address/pointer bytes come from the ADS1115; PIC ACK & PIC NAck are sent by the master after received bytes.

Bit	Field	Type	Reset	Description
7:2	RESERVED	W	000000b	Always write 000000b
1:0	P[1:0]	W	00b	**Register address pointer** 00b : Conversion register 01b : Config register 10b : Lo_thresh register 11b : Hi_thresh register

Table 2-4 Register Pointer table.

ART
11/2/26

Experiment 10 I²C Communications

ART
11/3/26

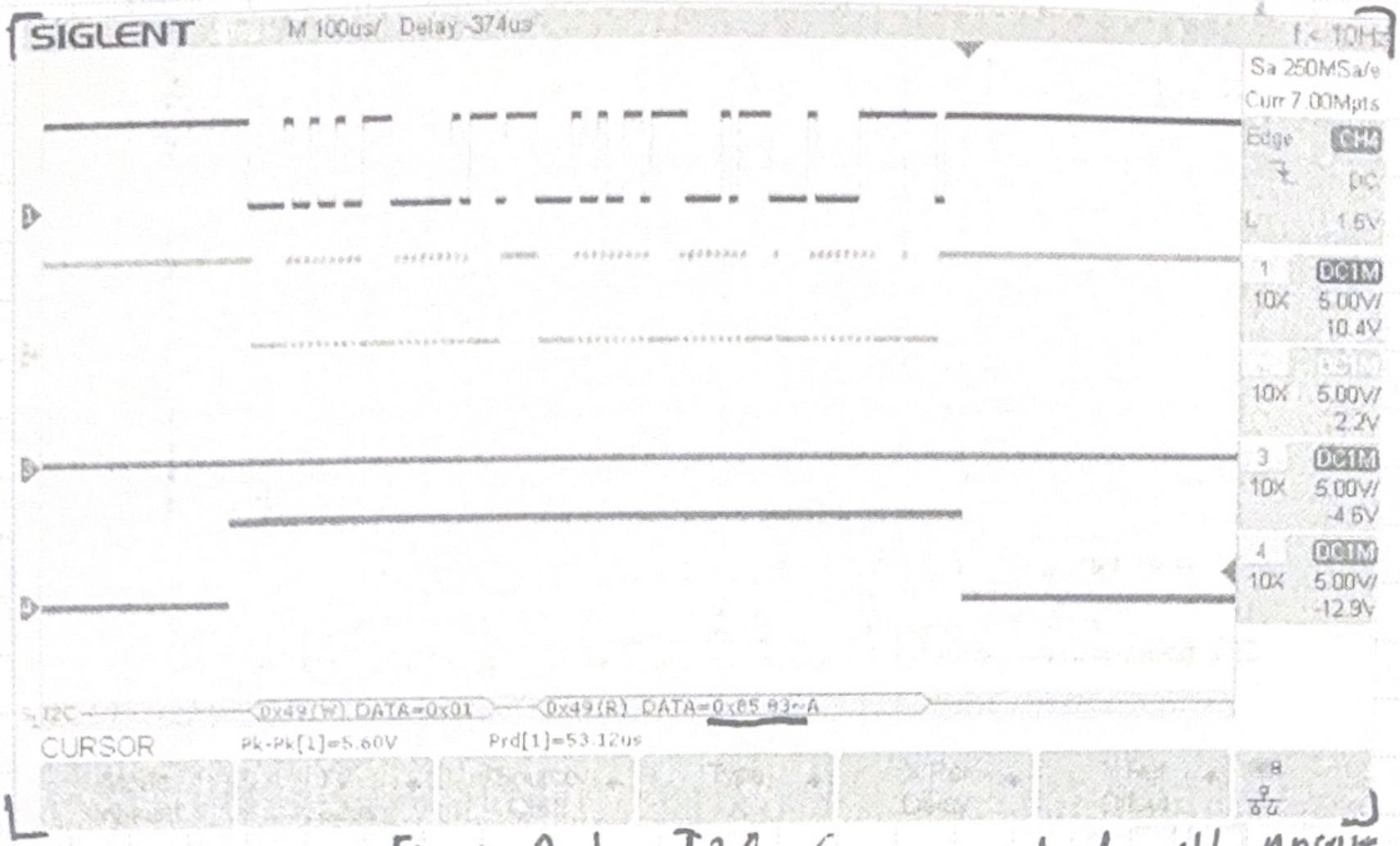


Figure 2-6 I2C Communicated with ADS1115 to read config registers

Byte	Bits	Fields
0x85 = 1000 0101	15	OS = 1 (idle)
	14:12	MUX = 000 (AIN0 – AIN1)
	11:9	PGA = 010 (±2.048 V)
	8	MODE = 1 (single-shot)
0x83 = 1000 0011	7:5	DR = 100 (128 SPS)
	4	COMP_MODE = 0
	3	COMP_POL = 0
	2	COMP_LAT = 0
	1:0	COMP_QUE = 11 (comparator disabled)

*Data sheet explanations of the two byte Config register

3. Configure the ADC. Now use the Config register descriptions to determine the two bytes required for:
   - AIN0 relative to GND
   - ± 6.144 V Full-scale range
   - Continuous conversion mode
   - 128 Samples per second
   - Comparator disabled

Document each chosen field & derive the complete 16-bit value yourself. Write it using this transaction:

```
Configure ADC (once at startup):
START → ADDR+W → ACK → 0x01 → ACK → [Config MSB] → ACK → [Config LSB] → ACK → STOP
```

Run the config write once after MCLR reset, then enter a do nothing loop for its Single Sequence capture. Read Config again to verify the settings, also once, using the read transaction from step 2; the write already selected config. Do not repeat the Config write on every ADC read.

AKT
11/3/26

Experiment 10 I²C Communications

ART
11/3/26

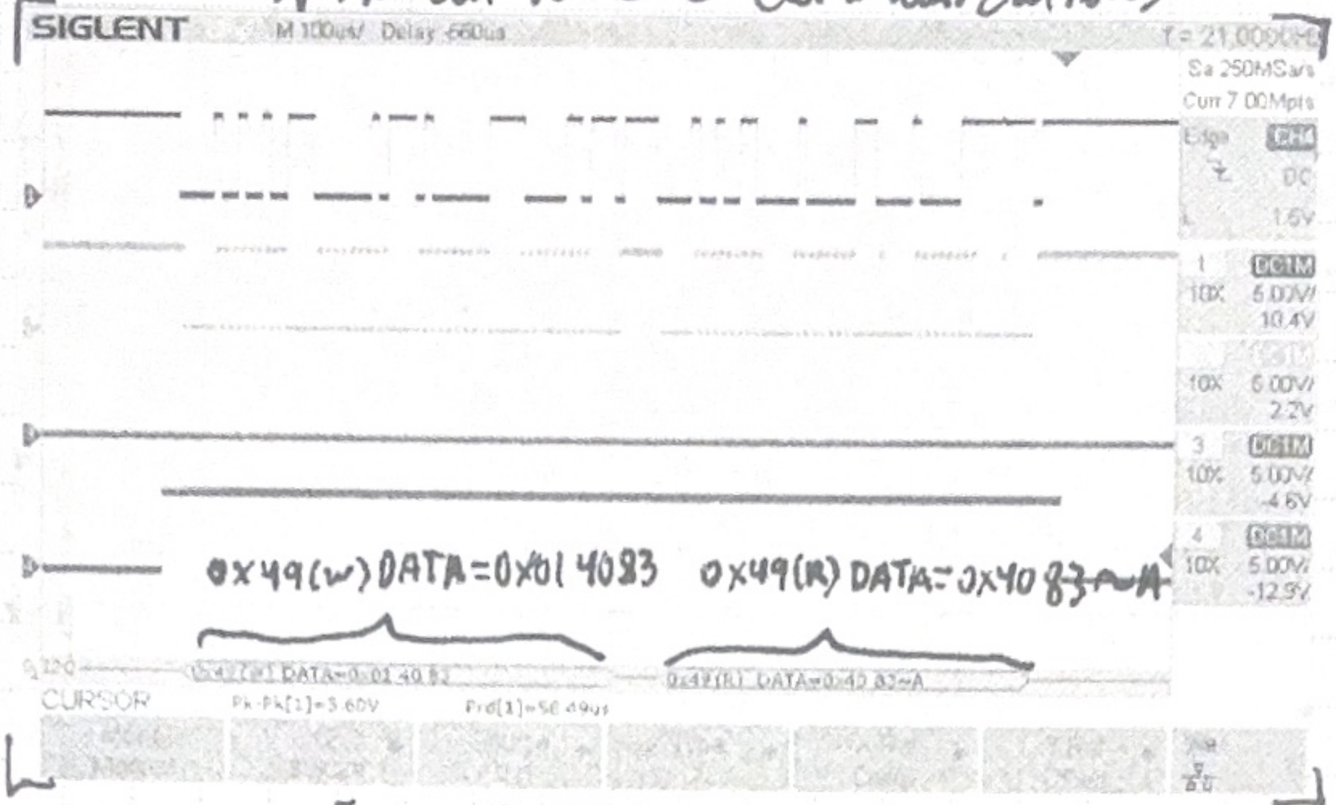


Figure 2-7 Configured ADS1115 as instructed (pg 268)

Bit	Field	Type	Reset	Description
15	OS	R/W	1b	**Operational status or single-shot conversion start** This bit determines the operational status of the device. OS can only be written when in power-down state and has no effect when a conversion is ongoing. When writing: 0b : No effect 1b : Start a single conversion (when in power-down state) When reading: 0b : Device is currently performing a conversion. 1b : Device is not currently performing a conversion.
14:12	MUX[2:0]	R/W	000b	**Input multiplexer configuration (ADS1115 only)** These bits configure the input multiplexer. *These bits serve no function on the ADS1113 and ADS1114. ADS1113 and ADS1114 always use inputs AIN_P = AIN0 and AIN_N = AIN1.* 000b : AIN_P = AIN0 and AIN_N = AIN1 (default) 001b : AIN_P = AIN0 and AIN_N = AIN3 010b : AIN_P = AIN1 and AIN_N = AIN3 011b : AIN_P = AIN2 and AIN_N = AIN3 100b : AIN_P = AIN0 and AIN_N = GND 101b : AIN_P = AIN1 and AIN_N = GND 110b : AIN_P = AIN2 and AIN_N = GND 111b : AIN_P = AIN3 and AIN_N = GND
11:9	PGA[2:0]	R/W	010b	**Programmable gain amplifier configuration** These bits set the FSR of the programmable gain amplifier. *These bits serve no function on the ADS1113. ADS1113 always uses FSR = ±2.048V.* 000b : FSR = ±6.144V(1) 001b : FSR = ±4.096V(1) 010b : FSR = ±2.048V (default) 011b : FSR = ±1.024V 100b : FSR = ±0.512V 101b : FSR = ±0.256V 110b : FSR = ±0.256V 111b : FSR = ±0.256V
8	MODE	R/W	1b	**Device operating mode** This bit controls the operating mode. 0b : Continuous-conversion mode 1b : Single-shot mode or power-down state (default)
7:5	DR[2:0]	R/W	100b	**Data rate** These bits control the data rate setting. 000b : 8SPS 001b : 16SPS 010b : 32SPS 011b : 64SPS 100b : 128SPS (default) 101b : 250SPS 110b : 475SPS 111b : 860SPS
4	COMP_MODE	R/W	0b	**Comparator mode (ADS1114 and ADS1115 only)** This bit configures the comparator operating mode. *This bit serves no function on the ADS1113.* 0b : Traditional comparator (default) 1b : Window comparator
3	COMP_POL	R/W	0b	**Comparator polarity (ADS1114 and ADS1115 only)** This bit controls the polarity of the ALERT/RDY pin. *This bit serves no function on the ADS1113.* 0b : Active low (default) 1b : Active high
2	COMP_LAT	R/W	0b	**Latching comparator (ADS1114 and ADS1115 only)** This bit controls whether the ALERT/RDY pin latches after being asserted or clears after conversions are within the margin of the upper and lower threshold values. *This bit serves no function on the ADS1113.* 0b : Nonlatching comparator. The ALERT/RDY pin does not latch when asserted (default). 1b : Latching comparator. The asserted ALERT/RDY pin remains latched until conversion data are read by the controller or an appropriate SMBus alert response is sent by the controller. The device responds with an address, and is the lowest address currently asserting the ALERT/RDY bus line.
1:0	COMP_QUE[1:0]	R/W	11b	**Comparator queue and disable (ADS1114 and ADS1115 only)** These bits perform two functions. When set to 11, the comparator is disabled and the ALERT/RDY pin is set to a high-impedance state. When set to any other value, the ALERT/RDY pin and the comparator function are enabled, and the set value determines the number of successive conversions exceeding the upper or lower threshold required before asserting the ALERT/RDY pin. *These bits serve no function on the ADS1113.* 00b : Assert after one conversion 01b : Assert after two conversions 10b : Assert after four conversions 11b : Disable comparator and set ALERT/RDY pin to high-impedance (default)

Table 2-5 Config Registers ADS1115

ART
11/3/26

Experiment 10 I^2C Communication

ART 11/3/26

4. Read ADC conversions. Connect a potentiometer across ADS1115 VDD & GND, with its wiper at A1N0. Do not exceed the ADC's supply voltage at its analog input. The ADS1115 now converts continuously on its own. The PIC reads the latest completed result only when it requests it. If the ADS1115 has lost power since step 3, Configure it once before starting the read loop. Start with the timer interval from part 1; you will choose a final interval in step 5. Each PIC polling cycle repeats both transactions:

```
Select Conversion:
START → ADDR+W → ACK → 0x00 → ACK → STOP

Read Conversion:
START → ADDR+R → ACK → ADC MSB → PIC ACK → ADC LSB → PIC NACK → STOP

Repeat both transactions at the PIC timer interval.
```

The ADS1115 retains its register pointer, but for this lab send pointer 0x00 on every polling cycle. If an address or write-data byte is NACKed, stop sending data, attempt STOP, & preserve/report the error. On a timeout or bus-low error, report the error & do not assume STOP or bus recovery will succeed; do not continue the transaction. The master's NACK after the final received byte is normal. Combine the ADC bytes as a signed 16-bit result. For the assigned full-scale range:

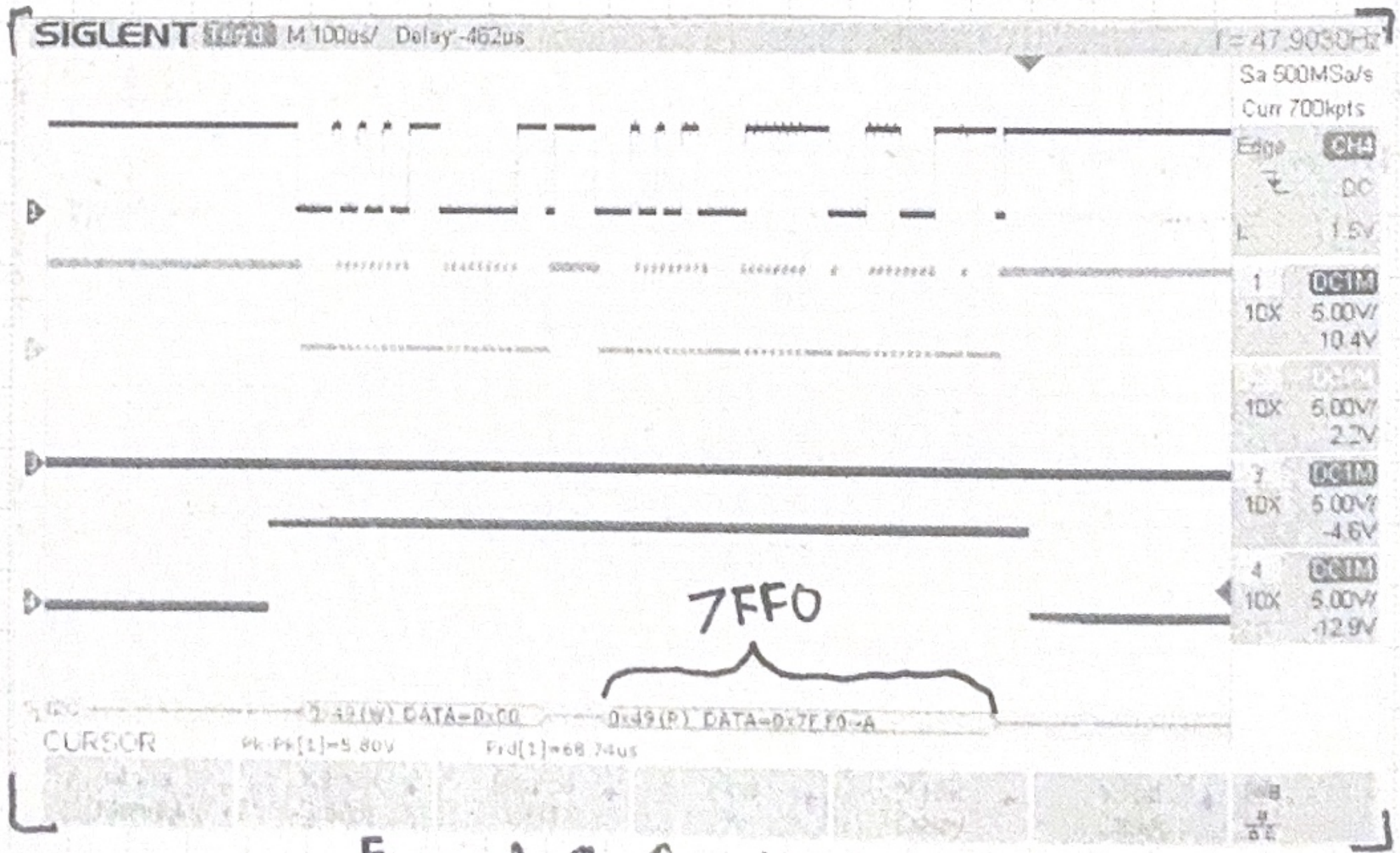
$$ADC_{16} = (MSB \ll 8) \mid LSB$$

$$V_{AIN0} = \frac{\text{Signed } ADC_{16}}{32768}(6.144\,V) \leftarrow VDD$$

For example, a positive result of 0x2000 = 8192 count gives:

$$V_{AIN0} = \frac{8192}{32768}(6.144\,V) = 1.536\,V$$

The ±6.144 V range determines ADC scaling; it does not raise the allowable input voltage above ADS1115 VDD.



High end 5V = 7FF0

Low end 0V = EF00

↑ Invert the Low end & add 1 to get the correct signed value = -4352 ↑ decimal

Figure 2-8 Constant ADC conversions

* This is the high end at 5V, meaning the above equation should have VDD where 6.144V is.

ART 10/4/26

5. Observe ALERT/RDY & choose a polling interval. Find Conversion Ready pin in the ADS1113 datasheet. Configure ALERT/RDY for conversion-ready pulses using the threshold registers & config fields. The output is open-drain; ensure it has a pull up to a compatible supply. 11/4/26

```
Set low threshold (once):
START → ADDR+W → ACK → 0x02 → ACK → [Lo MSB] → ACK → [Lo LSB] → ACK → STOP

Set high threshold (once):
START → ADDR+W → ACK → 0x03 → ACK → [Hi MSB] → ACK → [Hi LSB] → ACK → STOP

Enable conversion-ready output (once):
START → ADDR+W → ACK → 0x01 → ACK → [Updated Config MSB] → ACK → [Updated Config LSB] → ACK → STOP
```

Perform the ALERT/RDY setup writes only once, then leave the PIC in a do-nothing loop while the ADS1115 converts on its own. Trigger the O-scope on ALERT/RDY. Measure the interval between pulses & calculate the observed conversion rate. Compare it with the configured data rate. Choose & Justify PIC timer interval for repeted I²C reads. Explain whether that interval will retreive every conversion or skip some. Confirm the actual interval between I²C read cycles on the scope.

7.3.8 Conversion Ready Pin (ADS1114 and ADS1115 Only)

The ALERT/RDY pin can also be configured as a conversion-ready pin. Set the most-significant bit of the Hi_thresh register to 1b and the most-significant bit of Lo_thresh register to 0b to enable the pin as a conversion-ready pin. The COMP_POL bit continues to function as expected. Set the COMP_QUE[1:0] bits to any 2-bit value other than 11b to keep the ALERT/RDY pin enabled, and allow the conversion-ready signal to appear at the ALERT/RDY pin output. The COMP_MODE and COMP_LAT bits no longer control any function. When configured as a conversion-ready pin, ALERT/RDY continues to require a pullup resistor. The ADS111x provide an approximately 8µs conversion-ready pulse on the ALERT/RDY pin at the end of each conversion in continuous-conversion mode, as shown in Figure 7-8. In single-shot mode, the ALERT/RDY pin asserts low at the end of a conversion if the COMP_POL bit is set to 0b.

ADS1115 Datasheet requirements for Conversion Pin

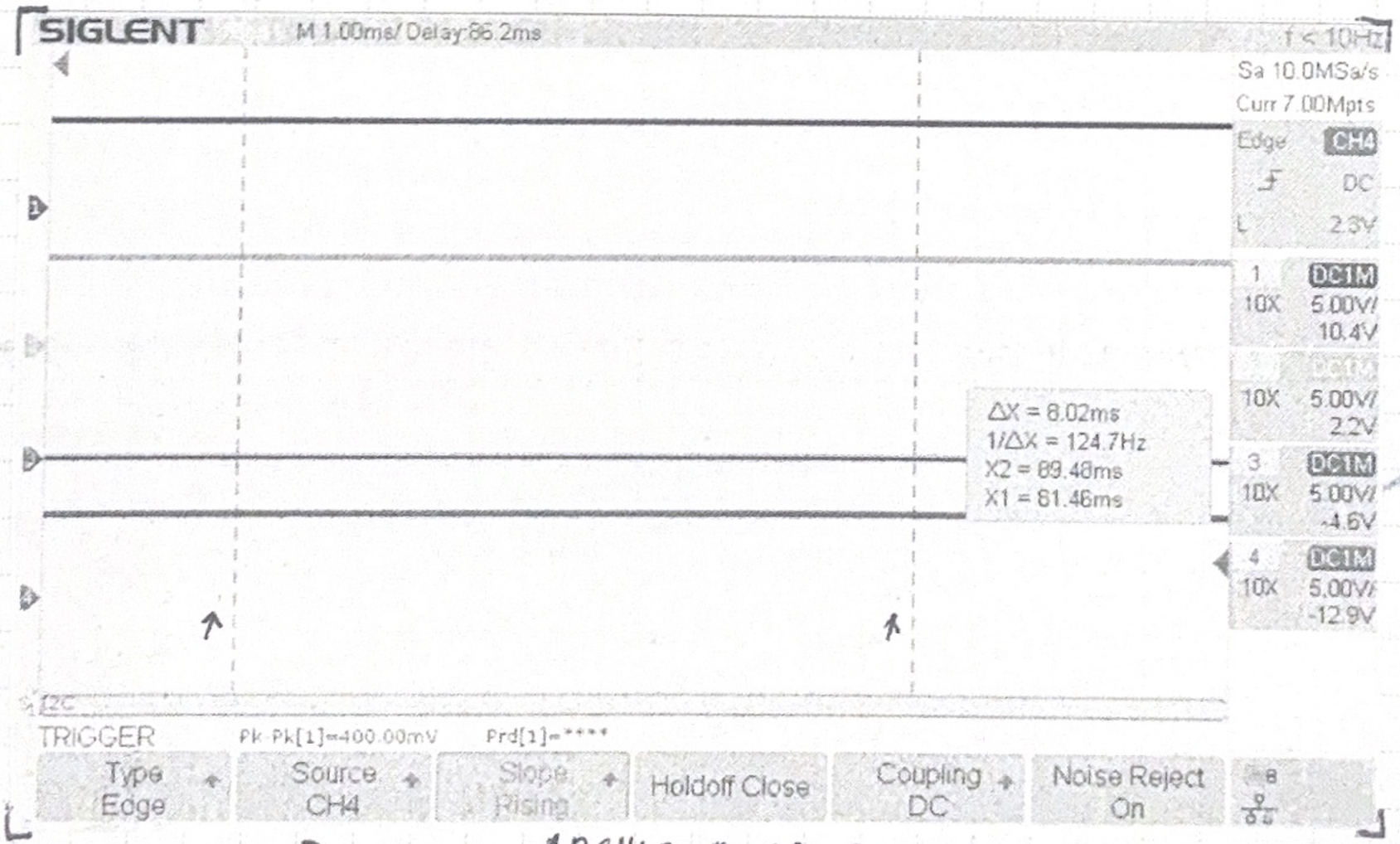


Figure 2-9 ADS1115 ALERT/RDY Conversion Pulse time

This configuration is set at 128 Samples per second which is 1 sample every 7.81ms. The measured rate matches this

ART
11/4/26

ART 11/4/26

Experiment 10 I²C Communication

6. Capture continuous reading. With your selected PIC timer interval, capture multiple consecutive conversion-pointer/write & Conversion read transactions at each potentiometer extreme. Measure AIN0 with a DMM at each setting & compare the measured voltage with the ADC result. Use CH1 = SDA & CH2 = SCL for decoding; Connect another Scope channel to ALERT/RDY for the timing test

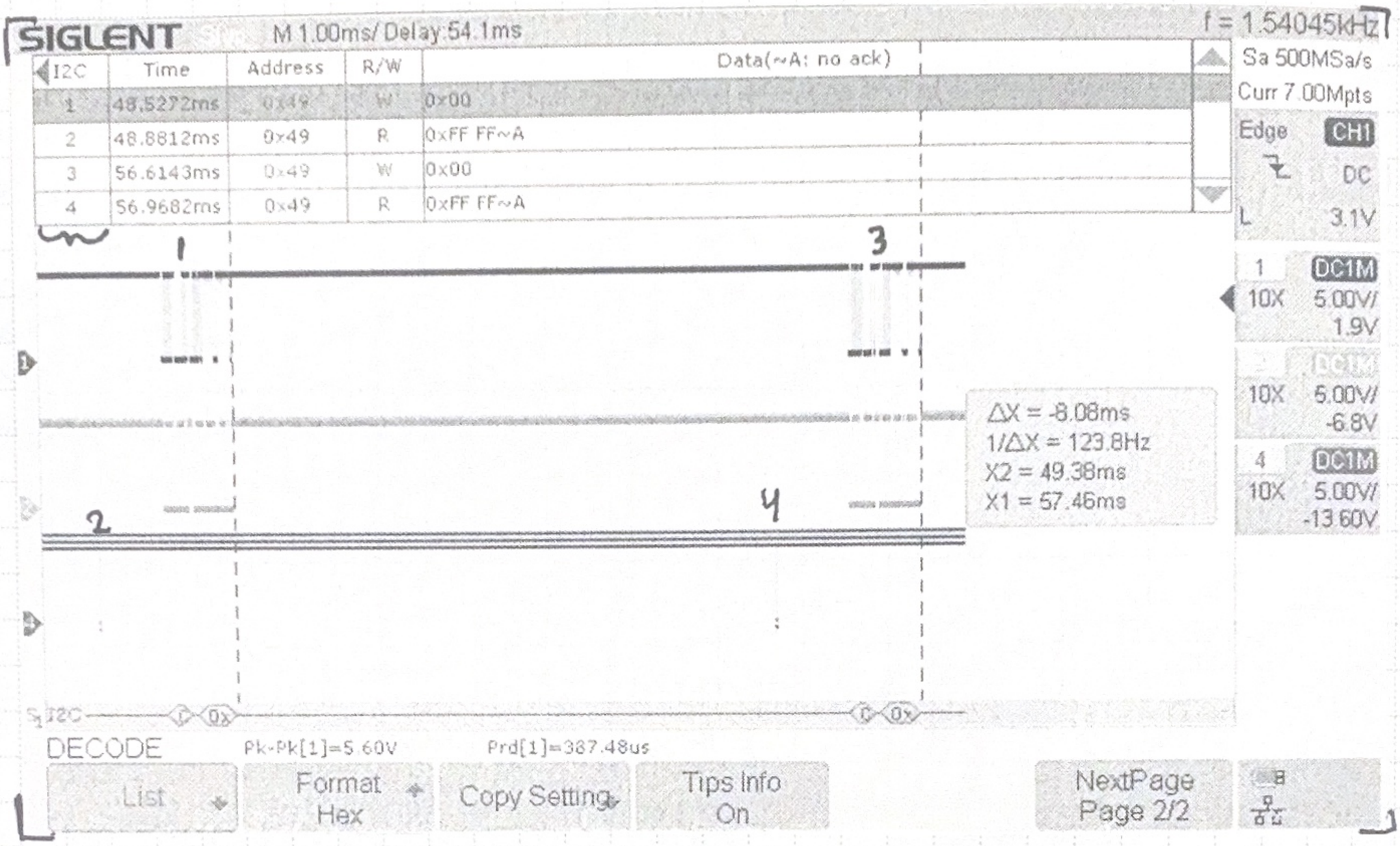


Figure 2-10 Measured constant ADC conversions on matched interval

TJR 11-4-26

*In the above capture you can see the time difference from 1&2 to 3&4 is 8ms

ART 11/4/26

Experiment 10 I^2C Communication

ART
11/4/26

Part 3 - PIC I^2C Slave: Inputs, Display, & ADC

Goal

Part 3 requires 2 PIC 16F883 programs: I^2C Master Code & I^2C Slave code, running on separate PICs. Develop the slave at 7-bit address 0x20 & extend your pt 2 master code to test the slave, read its digital inputs, write display characters, & retreive 10-bit ADC measurements. The slave uses a small register-pointer interface similar in concept to the ADS1115. The slave must normally service I^2C events promptly. Either interrupts or polling the MSSP event flag is acceptable. Keep hardware communication routines reusable & separate from application behavior.

Before Lab

- Bring your working Part1/Part2 master source code. For Part3, create a master MPLab X project & a separate slave MPLab X project. You will write, build program, & test both master & slave firmware throughout Part3. Select the correct Main Project before programming each PIC.
- Draw the complete two-PIC system schematic: RC3/SCL (Pin 14), RC4/SDA (Pin 15), common ground, & the effective SDA/SCL pull-ups. Account for pull-ups already installed on connected boards; do not assume every devices needs another pair.
- Use the PIC16F883 Datasheet to document MSSP 7-bit slave-mode setup, address matching, SSPBUF, ACK/NACK, receive/transmit events, SSPIF, CKP/clock stretching, SSPOV & WCOL. Explain which device controls the final ACK/NACK of a read & when the slave can hold SCL low.
- Prepare flowcharts for slave initialization & servicing an I^2C event. Include receive-address, receive data, transmit-data, & error paths. Extend these flowcharts as new behavior is introduced.

Suggested Software Organization (optional)

One approach is to separate the slave code into Level 1 (MSSP hardware events), Level 2 (Pointer & read/write transactions), & Level 3 (PortB, display, ADC). The three-level I^2C slave design guide provides optional flowcharts, pseudocode, & transaction examples. The master code can reuse your Part 2 I^2C routines, with new application logic to send each test transaction & receive its results. This is a suggested design, not required file layout. Each PIC project may use one PIC-AS .S file or multiple .S modules if responsibilities are clear. Do not put executable routines in .inc files.

In the Lab

For every step, develop & test both sides of the I^2C link: the master must initiate the required transactions & processes any returned data or status; the slave must recognize its address & handle the corresponding requests. Do not submit only the slave firmware or rely on an unchanged Part 2 master for the completed Part 3 assignment.

ART
11/5/26

Experiment 10 I²C Communication

ART 11/5/26

1. Verify the Slave address. Assemble the minimum slave circuit & connect it to the existing master SDA/SCL bus. Confirm the slave PIC is programmed & powered. Configure its MSSP for the 7-bit address 0X20.
Use the partl master address probe, first with the assigned address & then with an unrelated address. Capture an ACK for the correct address & a NACK for the other address. The 7-bit address & complete transmitted bytes are different:

START → 0x20 + W (0x40) → slave ACK → STOP
START → other address + W → NACK → STOP

*Sending data from the master follows exactly the same process as before as shown above. Configuring the slave properly means it should handle deciding if the Address was correct & sending the correct response on its own. The code only needs to detect if the received byte was an address, the state of the R/W bit & clearing the receive buffer.

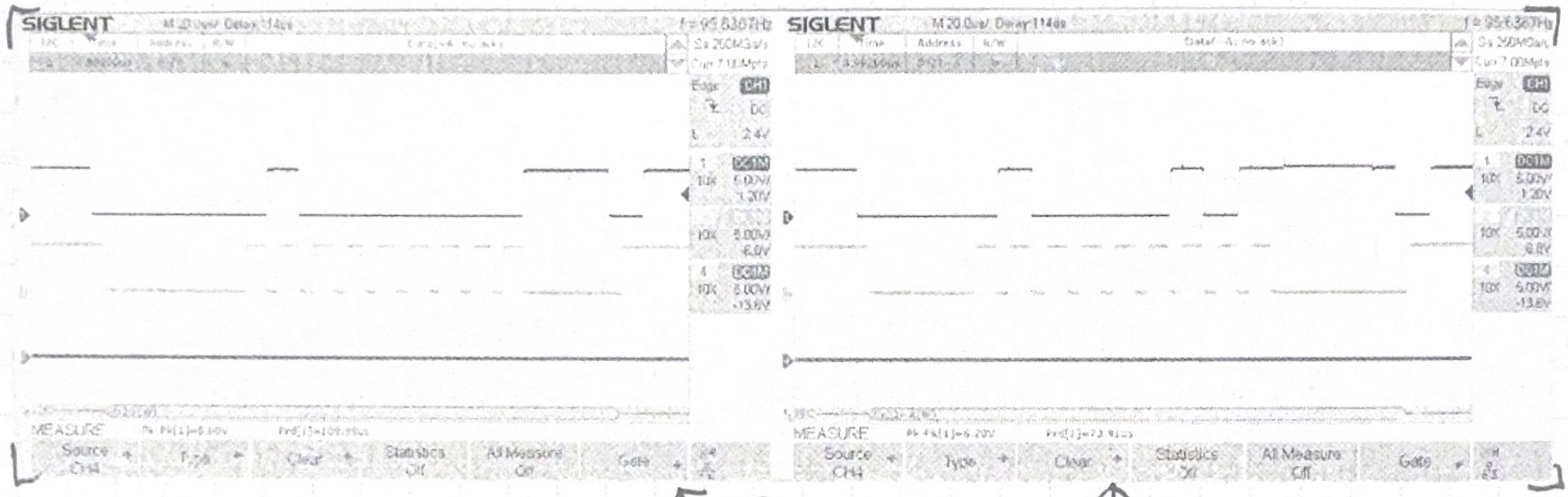


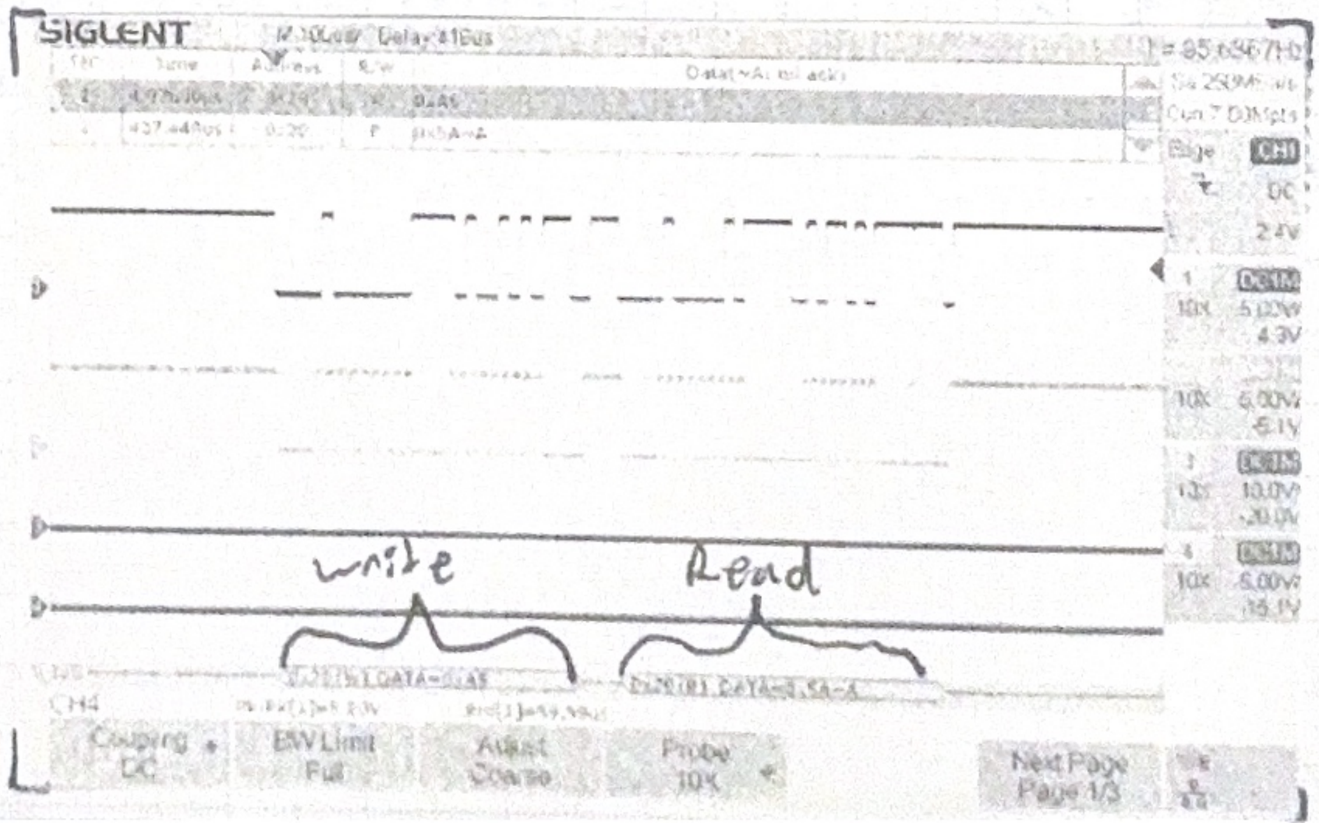
Figure 3-1 PIC Slave Address ACK & NACK

2. Test one received byte & one transmitted byte. Extend the slave to accept a byte after its address & modify the master to perform both tests. First write 0xA5 & check both slave ACK's. Next have the slave return a fixed byte 0x5A to a master read. For the one-byte read, the master sends NACK after receiving the byte.

Write test: START → 0x40 → ACK → 0xA5 → ACK → STOP
Read test: START → 0x41 → ACK → 0x5A → Master NACK → STOP

Keep these tests small. Verify them on the O-scope before adding the register-pointer logic.

Figure 3-2 Write & Read Fixed test



ART 11/5/26

3. Add the register-pointer interface. A slave write begins with a pointer byte. The pointer selects a logical register; Optional following bytes write that register. A later master read returns data from the selected register. Retain the selected pointer after STOP Sequences. For an initial diagnostic, temporarily echo the selected pointer on a read. Test at least two different values such as 0xA5 & 0x3C. Then implement the following application register map:

Pointer	Function	Direction	Data
0x00	PORTB switches	Read	One byte, RB5:RB0
0x01	DLG7137 display	Write	One 7-bit ASCII character
0x02	PIC ADC at AN2	Read	Two bytes, high then low
0x04	Device status, including I²C status	Read	One byte

Document how your code handles an unsupported pointer or number of Data bytes

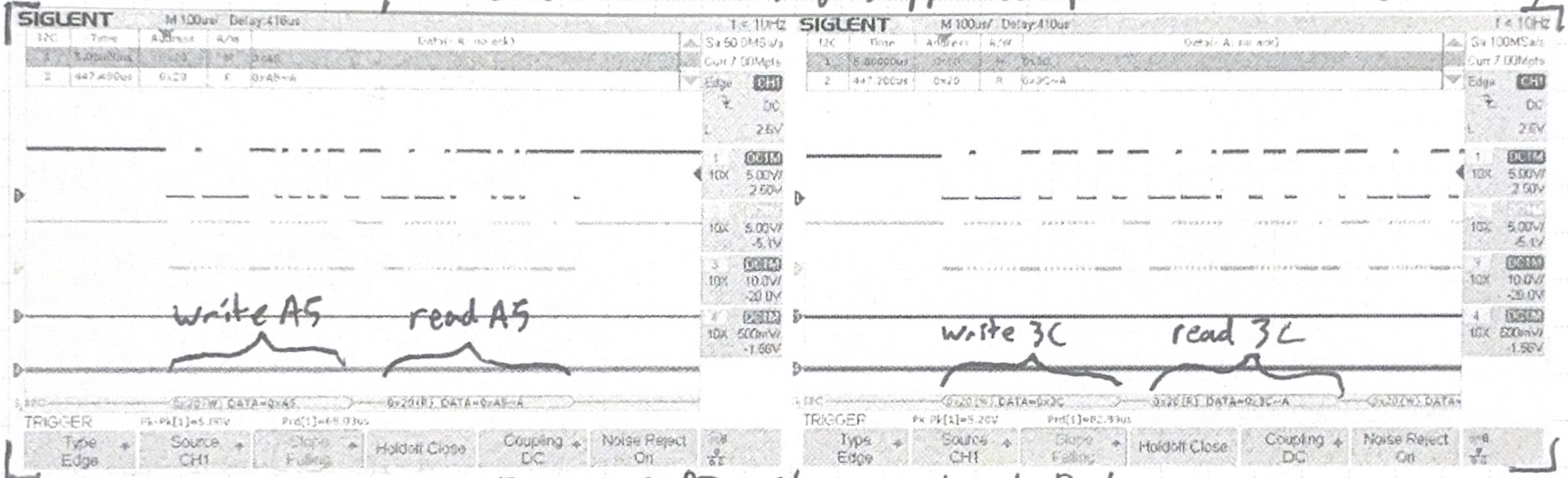


Figure 3-3 PIC slave echoed Byte

4. Read switches & Status. Connect six DIP switches to Slave PortB RB0-RB5 to GND. Enable suitable pull ups so an open switch reads 1 & a closed switch reads 0. Reserve RB6 & RB7 for PICkit programming. Return only bits 5:0 for pointer 0x00. Add pointer 0x04 as a one-byte device status register. Include I²C-related status or fault information, such as MSSP receive overflow or write collision. Choose your own status encoding & handling; document what each value means & how its cleared. You may also report other useful conditions, such as ADC conversion busy/ready state, invalid data, or application faults. Read the status register after normal transactions & explain results.

Select PORTB: START → 0x40 → ACK → 0x00 → ACK → STOP
Read PORTB: START → 0x41 → ACK → switch byte → Master NACK → STOP

Select status: START → 0x40 → ACK → 0x04 → ACK → STOP
Read status: START → 0x41 → ACK → status byte → Master NACK → STOP

Change several DIP-switch patterns. Compare the actual switch pattern positions, returned six-bit value, & O-scope decode. Keep the status read working as you add later features.

ART
11/6/26

Experiment 10 I²C Communication

ART 11/6/26

Clock-stretching experiment. After the PortB read works normally, add a short inline delay in the slave's PortB read path, while its MSSP hardware is holding SCL Low & before it releases SCL to send the data byte. Compare a normal read with several progressively longer delays, for example 0, 100µs, 1ms, & 10ms. Calculate or measure the actual delay; do not ~~measure~~ assume that a software delay produces clock stretching unless the scope shows SCL held Low.
Capture SDA & SCL & measure the extended SCL Low time for each delay. Does the master wait & receive the correct PortB byte? If the master has a timeout, ~~explain~~ determine approximately when it reports a failure & compare it with the measured stretch. If it has no timeout, explain how a stalled read behaves & how you regain control. Record the masters result for each test. Restore normal PortB operation before proceeding.

* This capture shows both the DIP switch status, the status register, & a slave clock stretch at 1ms.

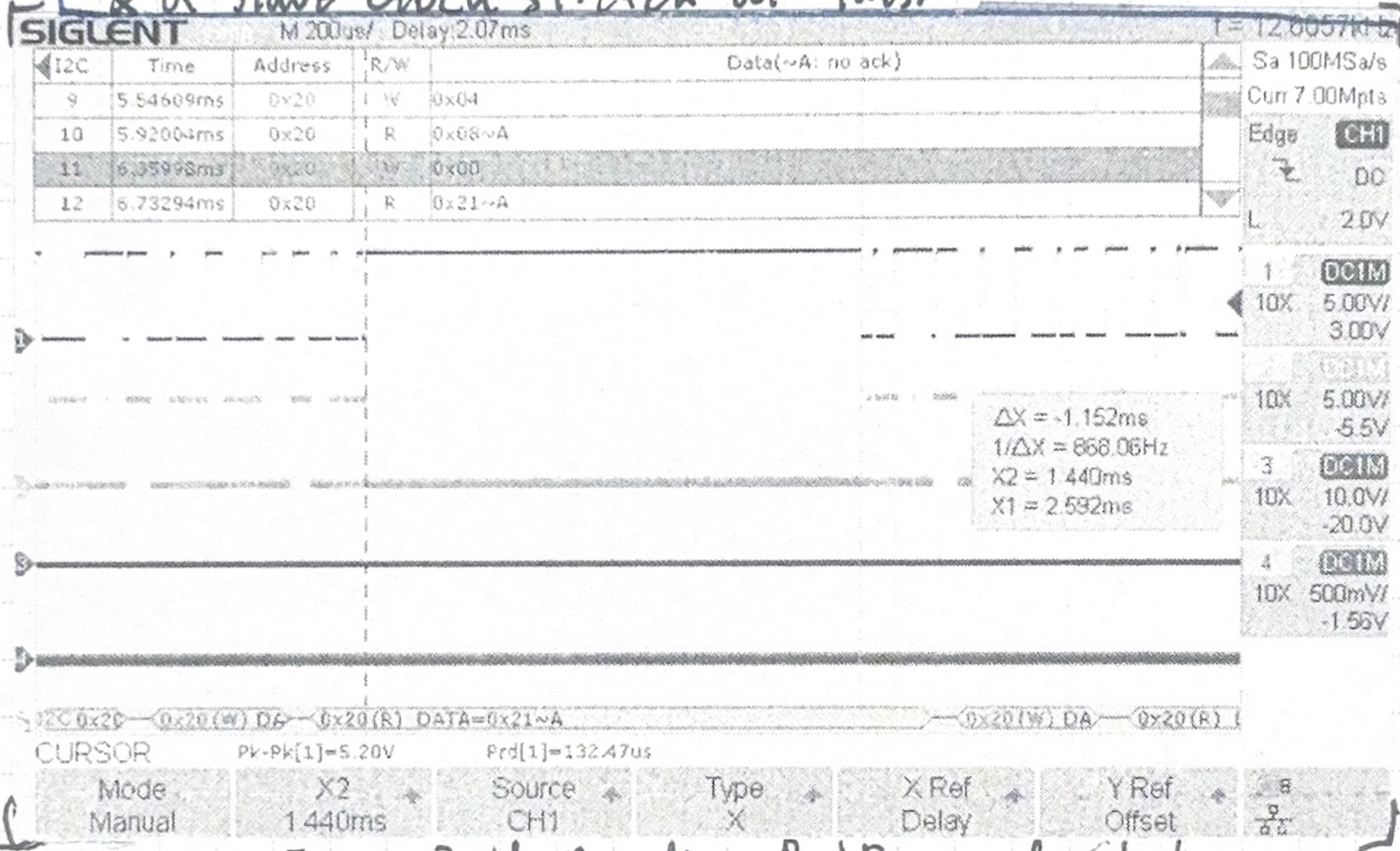


Figure 3-4 Reading PortB pins & Status on PIC slave with Clock stretch = 1ms

5. Write a character to the DLG7137. Use the DLG7137 single-character display from Lab 2. It accepts seven-bit ASCII; the slave must drive its data lines & pulse its active-low WR input. RC3 & RC4 remain dedicated to I²C, so do not reuse the original PortC wiring unchanged. ~~Modified~~ Modify the DLG7137 wiring to use two available PortA pins. Document & explain the change. Check the DLG7137 data sheet for orientation, control pins, power, & write timing. Document the wiring & decoupling. A write to pointer 0x01 followed by ASCII 0x41 should display A

START → 0x40 → ACK → 0x01 → ACK → 0x41 → ACK → STOP

Stage the received character & update the display outside the time-sensative I²C event handler. Repeate with a different printable ASCII character. Confirm the PortB & status reads still work.

ART 11/9/26

ART
11/9/26

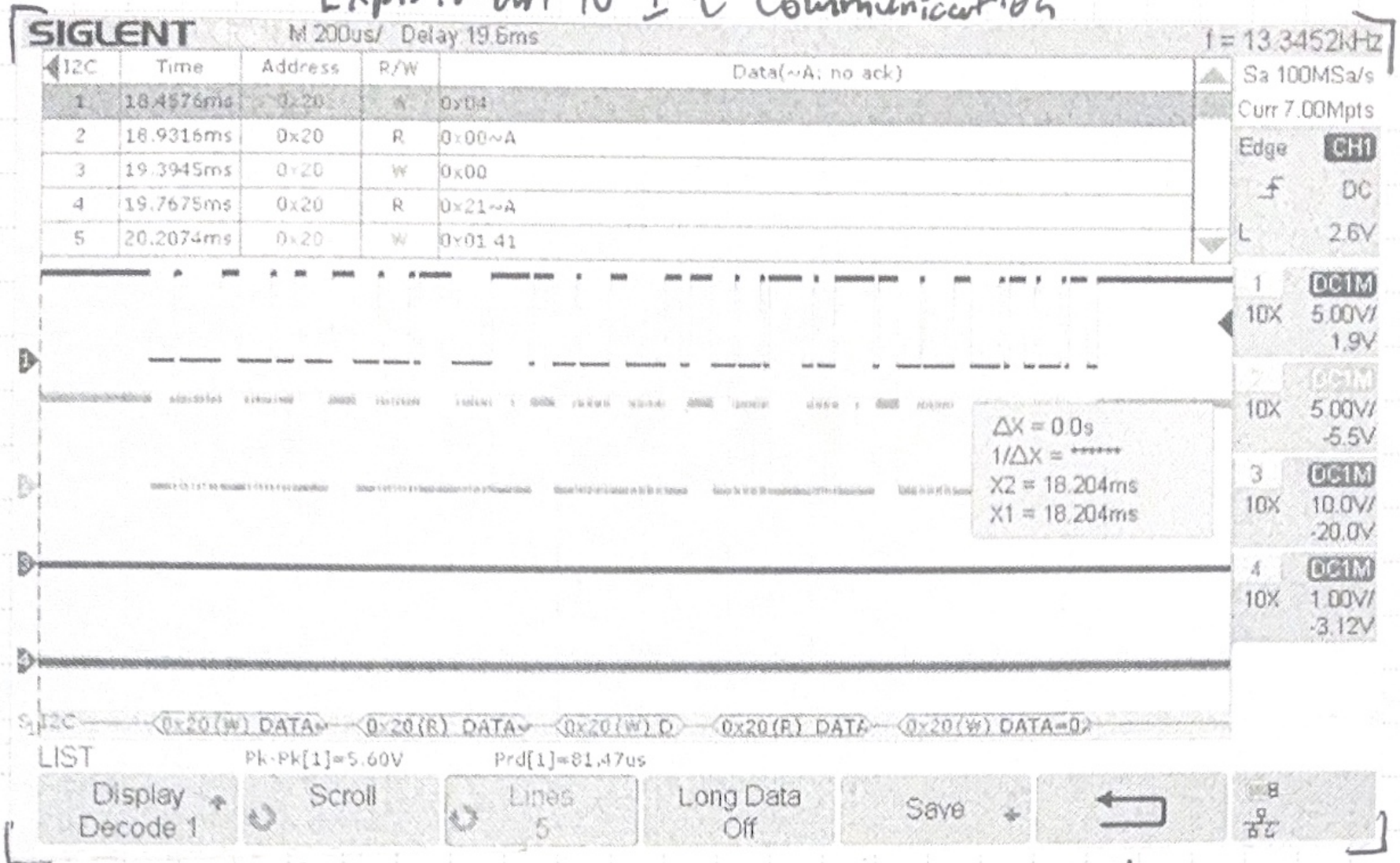


Figure3-5 Working Step 4 Code with IA Display Bent

b. Read the slaves ADC. Connect a potentiometer between the slaves VDD & GND, with the wiper on RA2/AN2 (pin4). Configure the PIC's 10-Bit ADC, VDD/VSS references, & right Justified result. Use a timer to sample periodically without waiting in a long blocking loop. Pointer 0x02 returns a consistent two-byte sample: Upper two ADC bits in the first byte, lower eight bits in the second. Store conversions in RAM & ensure that I²C read bytes Cannot come from different ADC Conversions.

Select ADC: START → 0x40 → ACK → 0x02 → ACK → STOP Read ADC: START → 0x41 → ACK → ADC high → Master ACK → ADC low → Master NACK → STOP

Combine the received bytes & estimate the input voltage using the measured supply voltages:

$$ADC_{10} = ((high \,\&\, 0x03) << 8) \mid Low$$

$$V_{AN2} \approx \left(ADC_{10}/1023\right) V_{DD}$$

Measure the actual AN2 voltage at GND, approximately midpoint, & VDD settings. Compare each measurement with the I²C result (approximately 0, 512, & 1023 counts, respectively). Allow time for a valid conversions after reset. Verify that the scope shows the masters' ACK between data bytes & NACK after the last byte.

ART
11/10/26

ART
10/10/26

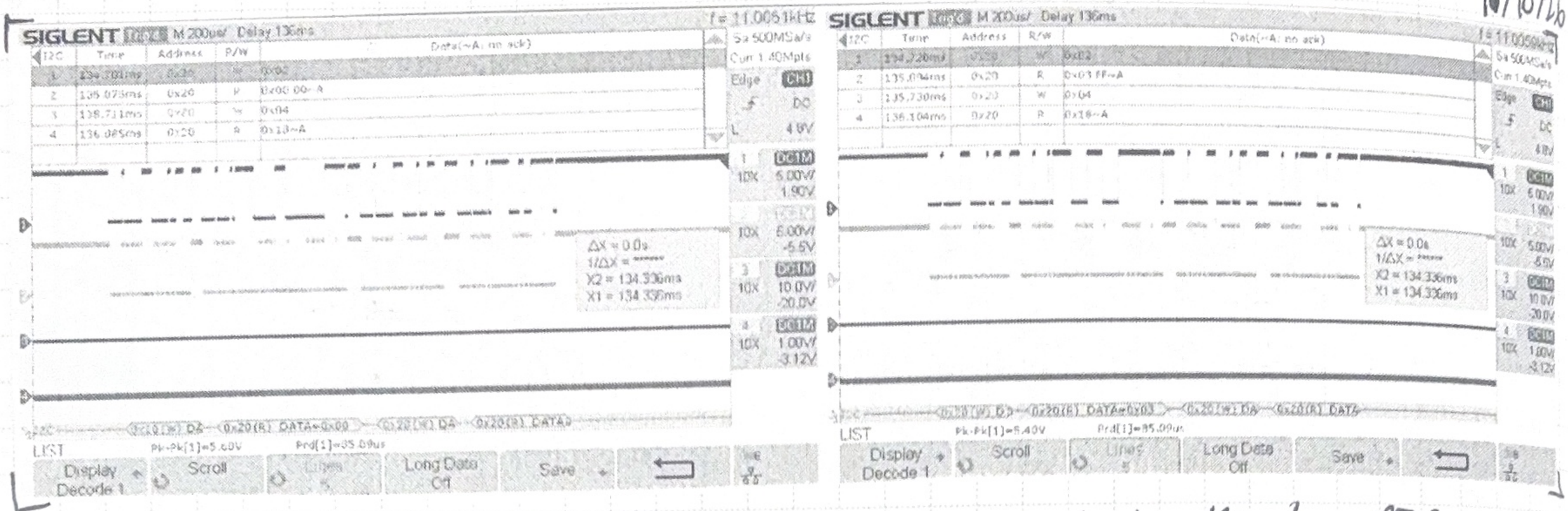


Figure 3-6 ADC PIC Slave Conversion read to Master PIC

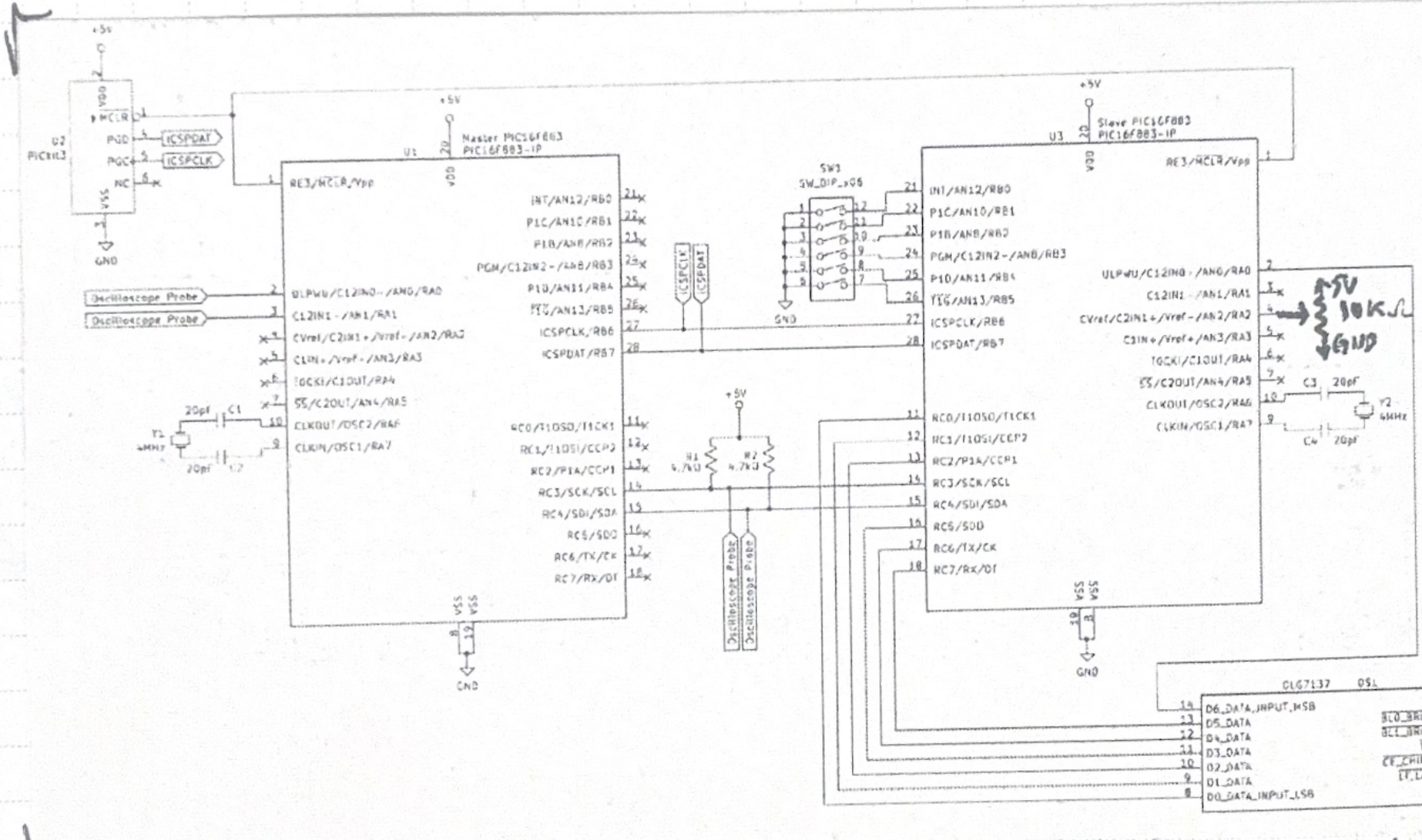


Figure 3-7 Complete Part3 Circuit

* This circuit is the culmination of all the parts of this section. For saftey, only add the sections as you do them in the lab, unless power codes have been done to ensure saftey.

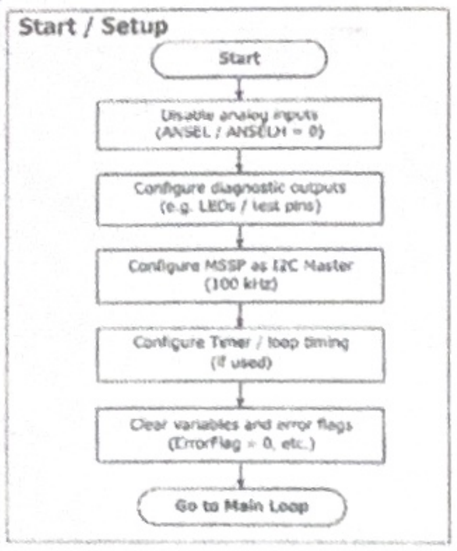
ART
10/10/26

Experiment 10 I²C Communication

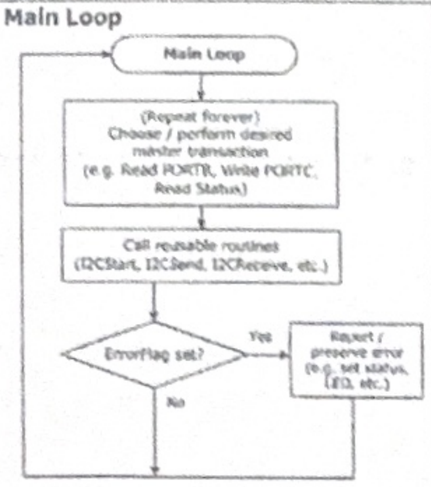
IVART
11/10/26

Lab 10 Part 3 - Master Firmware Flowcharts

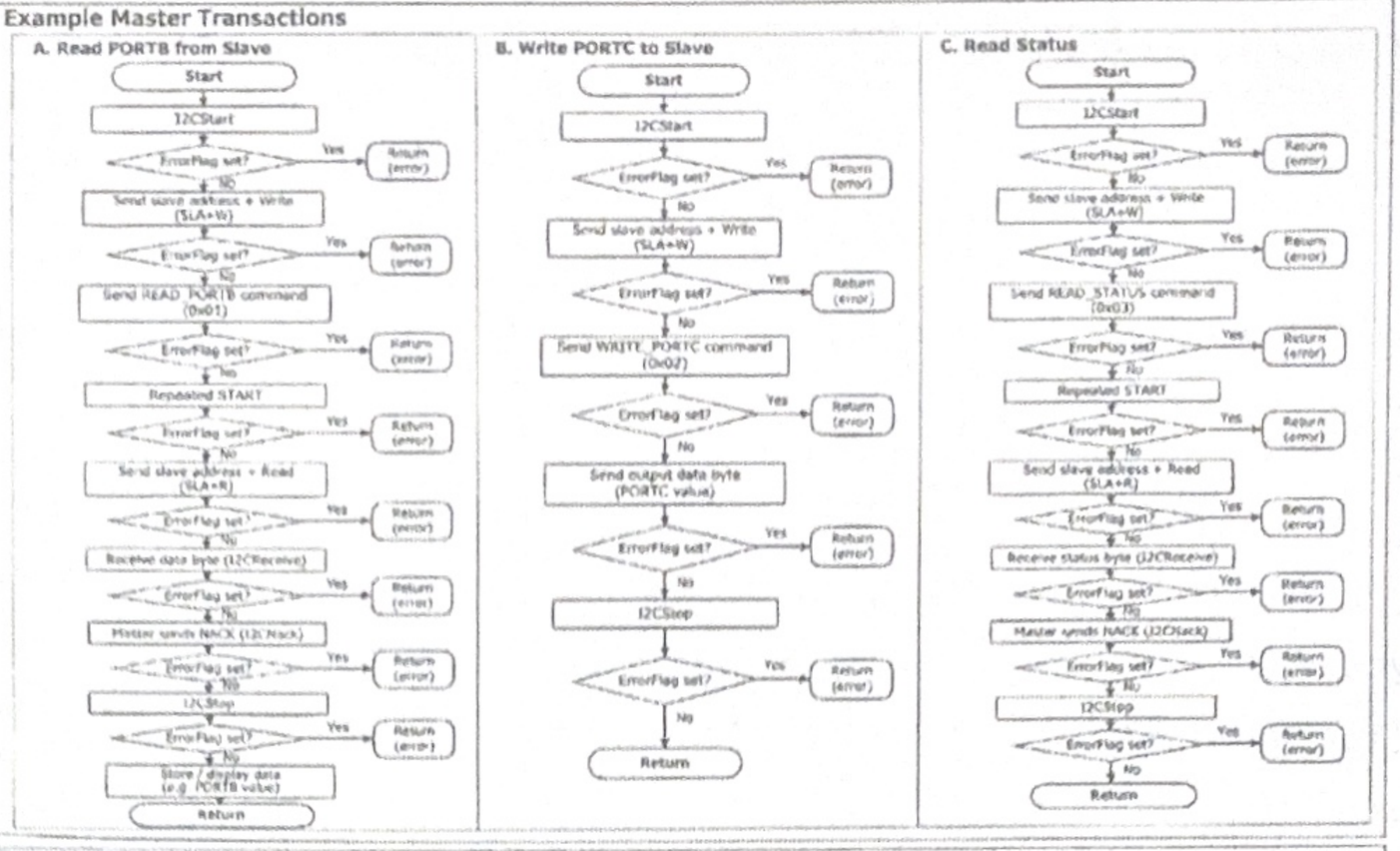
Start / Setup



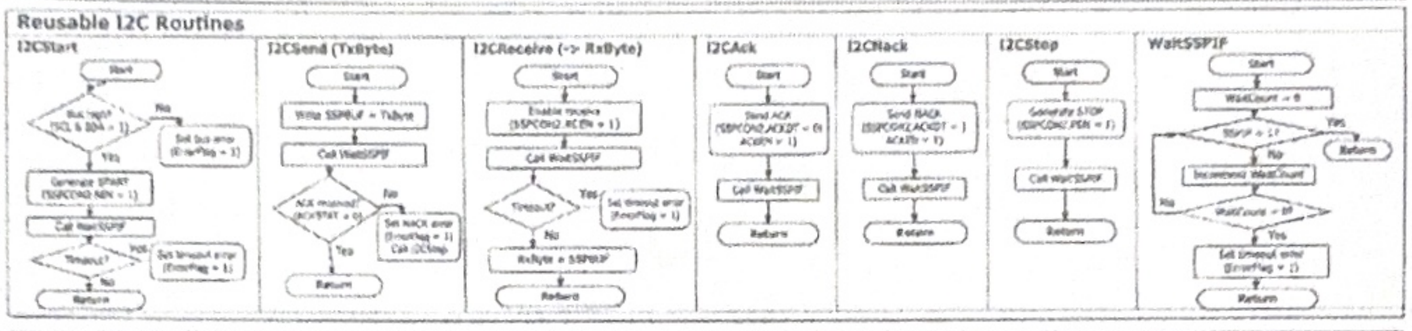
Main Loop



Example Master Transactions



Reusable I2C Routines



Example Slave Commands

READ_PORTB = 0x01 (read Slave PORTB)
WRITE_PORTC = 0x02 (write Slave PORTC)
READ_STATUS = 0x03 (read Slave status byte)

Master Variables

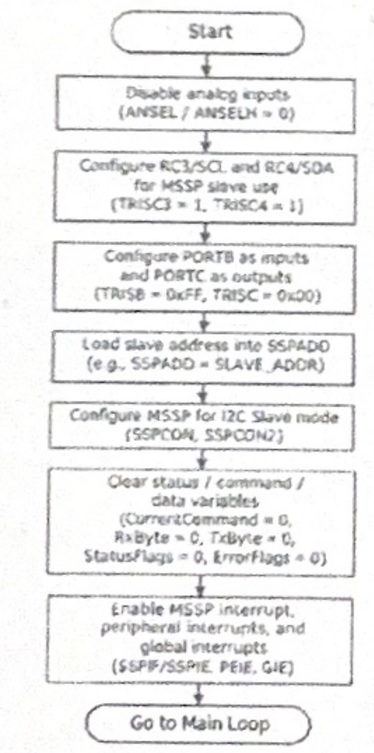
- ErrorFlag - error status flag (0 = no error, 1 = error)
- TxByte - byte to transmit (command, data, etc.)
- RxByte - received byte (data, status, etc.)
- TimerCount - general timer / loop counter (if used)
- WaitCount - timeout counter for I2C waits

I2C Notes

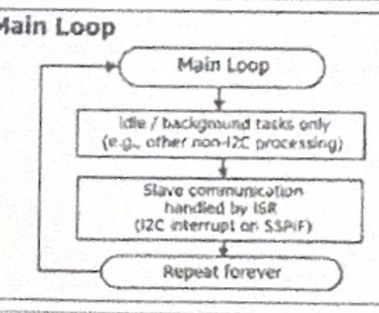
- PIC16F883 @ 4 MHz (Fosc = 4,000,000 Hz)
- MSSP in I2C Master mode, 100 kHz (SSPADD = 9 for 4 MHz)
- Reusable routines: I2CStart, I2CSend, I2CReceive, I2CAck, I2CNack, I2CStop, WaitSSPIF
- Errors may include: NACK (no acknowledge), timeout (SSPIF not set), or bus low (SCL/SDA stuck low)

Lab 10 Part 3 - Slave Firmware Flowcharts

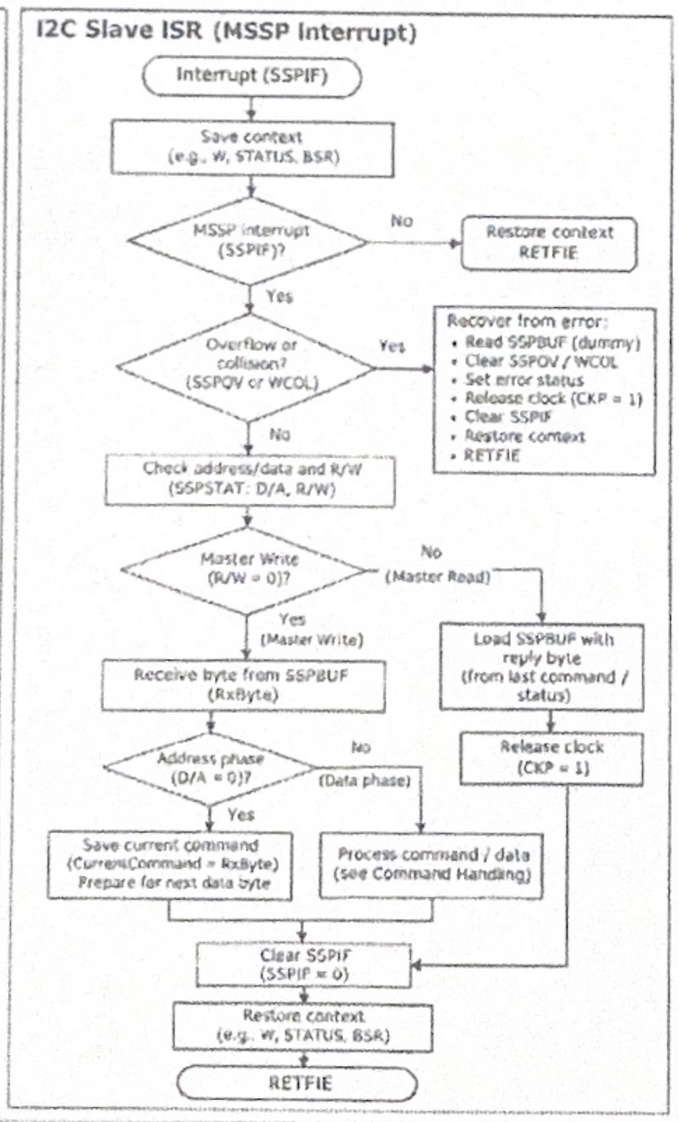
Start / Setup



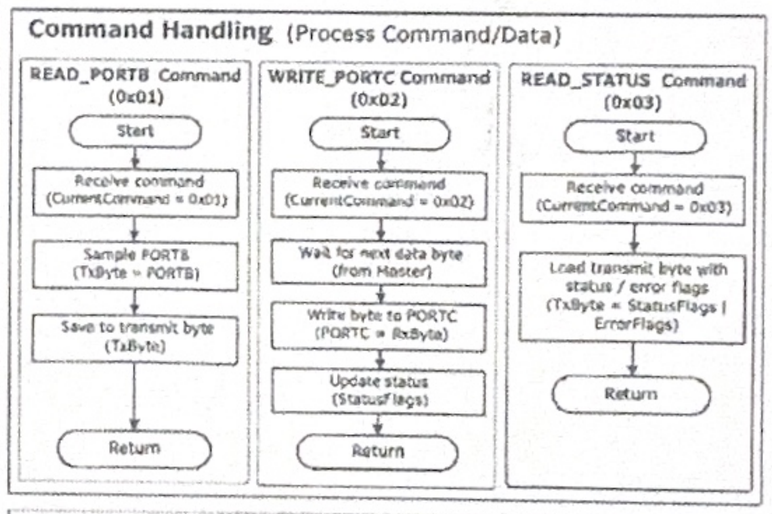
Main Loop



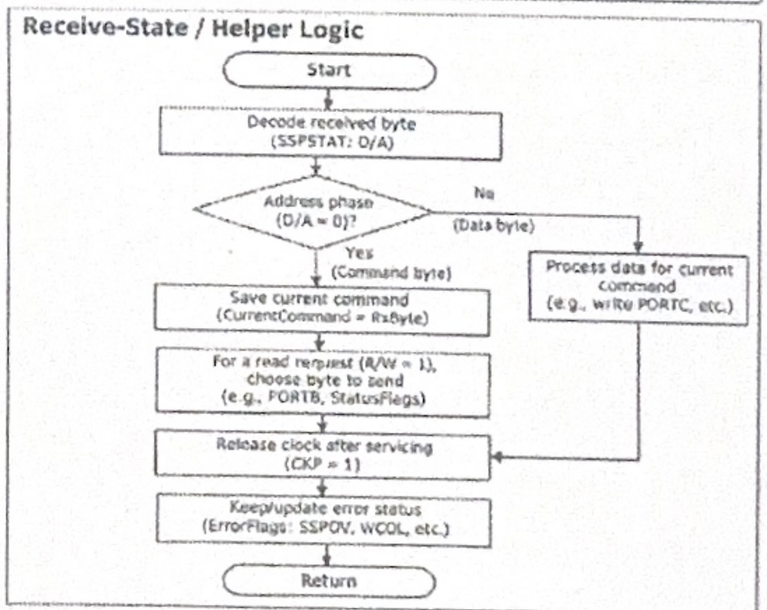
I2C Slave ISR (MSSP Interrupt)



Command Handling (Process Command/Data)



Receive-State / Helper Logic



Example Slave Commands

READ_PORTB = 0x01 - Read PORTB (return PORTB value)
WRITE_PORTC = 0x02 - Write next data byte to PORTC
READ_STATUS = 0x03 - Read status/error flags

Common Variables

- CurrentCommand - Last received command (0x01, 0x02, 0x03)
- RxByte - Most recent byte received from master
- TxByte - Byte to transmit to master
- StatusFlags - Slave status (e.g., command done, ready)
- ErrorFlags - Error flags (e.g., SSPOV, WCOL)

Slave Notes

- Interrupt-driven MSSP I2C slave (PIC16F883).
- PORTB readback (READ_PORTB) and PORTC write control (WRITE_PORTC).
- Status/error reporting (READ_STATUS).
- Recover from MSSP overflow (SSPOV) or write collision (WCOL).

Figures 3-8 & 3-9 Master & Slave PIC Flowcharts.

TJR
11-10-26

IART
10/10/26

AAT
11/10/26

Part 4 - UART-to-I²C Bridge

Goal

Turn the master PIC into a generic PC/UART-to-I²C bridge controlled by a student-built GUI sends custom byte sequences, reads responses, discovers I²C devices, & records communication. Test it with the Part3 PIC slave & Part2 ADS1115. Use the 4MHz master, 100kHz I²C, & 9600 baud 8N1 UART with Max 232 or equivalent RS-232/TTL interface. Support one host request at a time & ordinary 7-bit addresses. Separate STOP-terminated writes & reads are sufficient. Repeated START, general call, 10-bit addresses, & multi-master operation are not required.
The packet tables below specify the interface you must implement in part4. Choose your own routines, buffers, & error-handling structure while producing the specified externally visible behavior.

Before Lab

Follow the lab standard & Prepare:

- An updated schematic showing the PC serial adapter, Max232, Master PIC, both existing I²C devices, common ground, power, pull-ups, & compatible logic levels. Account for pull-ups on the ADS1115 breakout.
- Flowcharts for receiving a UART frame, validating the request, performing an I²C transaction, & returning the result. Document the EUSART & MSSP registers used.
- A command/response packet table & test cases for ACK/NACK, malformed requests, & transfer-length limits.
- A GUI plan covering hex/text entry, saved transmissions, live UART/I²C logs, a 128-address scan, & automatic bridge identification/reconnection.

Reuse the part3 master routines & UART framing from Lab8. Keep UART transport, I²C transactions, & bridge application behavior separate. Any clear PIC-AS source-file organization is acceptable.

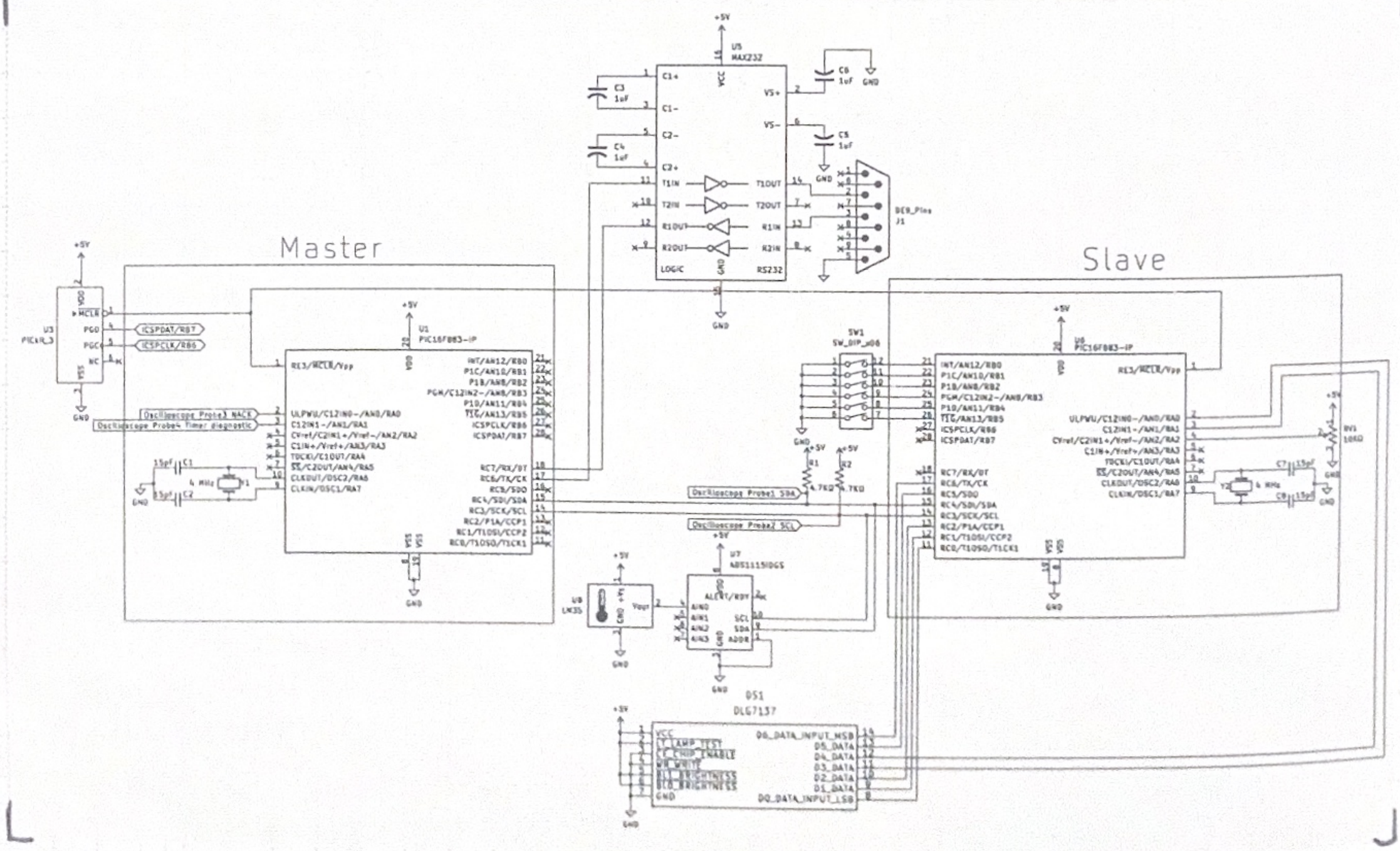


Figure 4-1 Full UART-to-I²C schematic

JART
12/11/26

Experiment 10 I²C Communication

UART packet format & error packets

UART packets contain binary bytes, shown here in hexadecimal. Each frame is 24, One command byte, a One-byte payload length, then that many payload bytes. Implement a maximum accepted payload of 24 bytes. The bridge must reject invalid lengths without overflowing its buffer or initiating I²C traffic.

Command byte	Operation	Host payload	Successful reply payload
49	Identity	None	33 75
50	Address probe	AA	00
57	Raw write	AA DD...	00
52	Raw read	AA NN	00 DD...
45	Error response only	None	Two error bytes (below)

Raw writes contain 1-23 data bytes; raw reads request 1-23 bytes. Successful replies include the full 24, command, length, & payload. On failure, return one error packet instead of a successful command reply. The bridge sends the host's write bytes unchanged & does not interpret I²C device registers. A pointer write & subsequent read are separate requests.

Preserve the Lab 8 UART framing & overrun error packets. Add separate I²C & bridge-validation error categories without reusing those codes. Every error packet has five bytes: 24 45 02, error category, detail.

Complete error packet	Meaning
24 45 02 01 00	UART framing error, retained from Lab 08
24 45 02 02 00	UART receive overrun, retained from Lab 08
24 45 02 04 SS	New I²C error; SS identifies the cause
24 45 02 05 EE	New bridge request/validation error; EE identifies the cause

I²C error detail (SS)	Meaning	Bridge error detail (EE)	Meaning
01	Address/data NACK	01	Invalid length or count
02	I²C timeout	02	Unknown command
03	Bus LOW / not idle	03	Invalid 7-bit address
04	SCL held LOW		

Preserve Lab 8's UART receiver recovery behavior: discard a damaged partial packet, clear the received fault, & accept the next valid request. If an I²C operation fails, stop/clean up the bus when possible, report the error & allow a later request. A request procedures one response: either its normal success packet or a five byte error packet.

ART
11/11/26

ART
11/11/26

Legend for all transaction tables

Symbol	Meaning
AA	Seven-bit I²C device address
NN	Number of bytes requested by a read
SS	I²C error detail
EE	Bridge error detail
WW	PORTB switch byte
HH	High result byte
LL	Low result byte
DD	One variable data byte
× N	Repeat the preceding byte/sequence N times (not transmitted)
START / STOP	I²C bus conditions
ACK / NACK	I²C ninth-bit responses, not data bytes

All other two-digit values are literal hexadecimal bytes. The Part3 PIC slave retains its register pointer across STOP, so a pointer write can be followed by a separate read.

Expected UART TX/RX & I²C transactions

UART TX: host → bridge. UART RX: bridge → host. The part3 PIC slave has assigned 7-bit address 20, so its I²C address bytes are 40 for write & 41 for read. The examples using 21 as an absent address assume no device responds at 21 on your bus; choose another unused ordinary address if necessary.

Test	UART TX	UART RX	Expected I²C transaction
Identify	24 49 00	24 49 02 33 75	None
Probe PIC, ACK	24 50 01 20	24 50 01 00	START, 40, ACK, STOP
Probe absent 21	24 50 01 21	24 45 02 04 01	START, 42, NACK, STOP
Display character 41	24 57 03 20 01 41	24 57 01 00	START, 40, ACK, 01, ACK, 41, ACK, STOP
Select PORTB	24 57 02 20 00	24 57 01 00	START, 40, ACK, 00, ACK, STOP
Read PORTB	24 52 02 20 01	24 52 02 00 WW	START, 41, ACK, WW, NACK, STOP
Select PIC ADC	24 57 02 20 02	24 57 01 00	START, 40, ACK, 02, ACK, STOP
Read PIC ADC	24 52 02 20 02	24 52 03 00 HH LL	START, 41, ACK, HH, ACK, LL, NACK, STOP

Each pointer write & subsequent read is a separate UART request & a separate I²C transaction, with STOP between them.
For the ADS1115 examples, 48 is a sample 7-bit address, with I²C address bytes 90 (write) & 91 (read). Substitute your verified Part2 address & its corresponding address bytes.

Test	UART TX	UART RX	Expected I²C transaction
Probe ADS1115	24 50 01 48	24 50 01 00	START, 90, ACK, STOP
Select Config	24 57 02 48 01	24 57 01 00	START, 90, ACK, 01, ACK, STOP
Read Config	24 52 02 48 02	24 52 03 00 HH LL	START, 91, ACK, HH, ACK, LL, NACK, STOP
Select Conversion	24 57 02 48 00	24 57 01 00	START, 90, ACK, 00, ACK, STOP
Read Conversion	24 52 02 48 02	24 52 03 00 HH LL	START, 91, ACK, HH, ACK, LL, NACK, STOP

ART
11/11/26

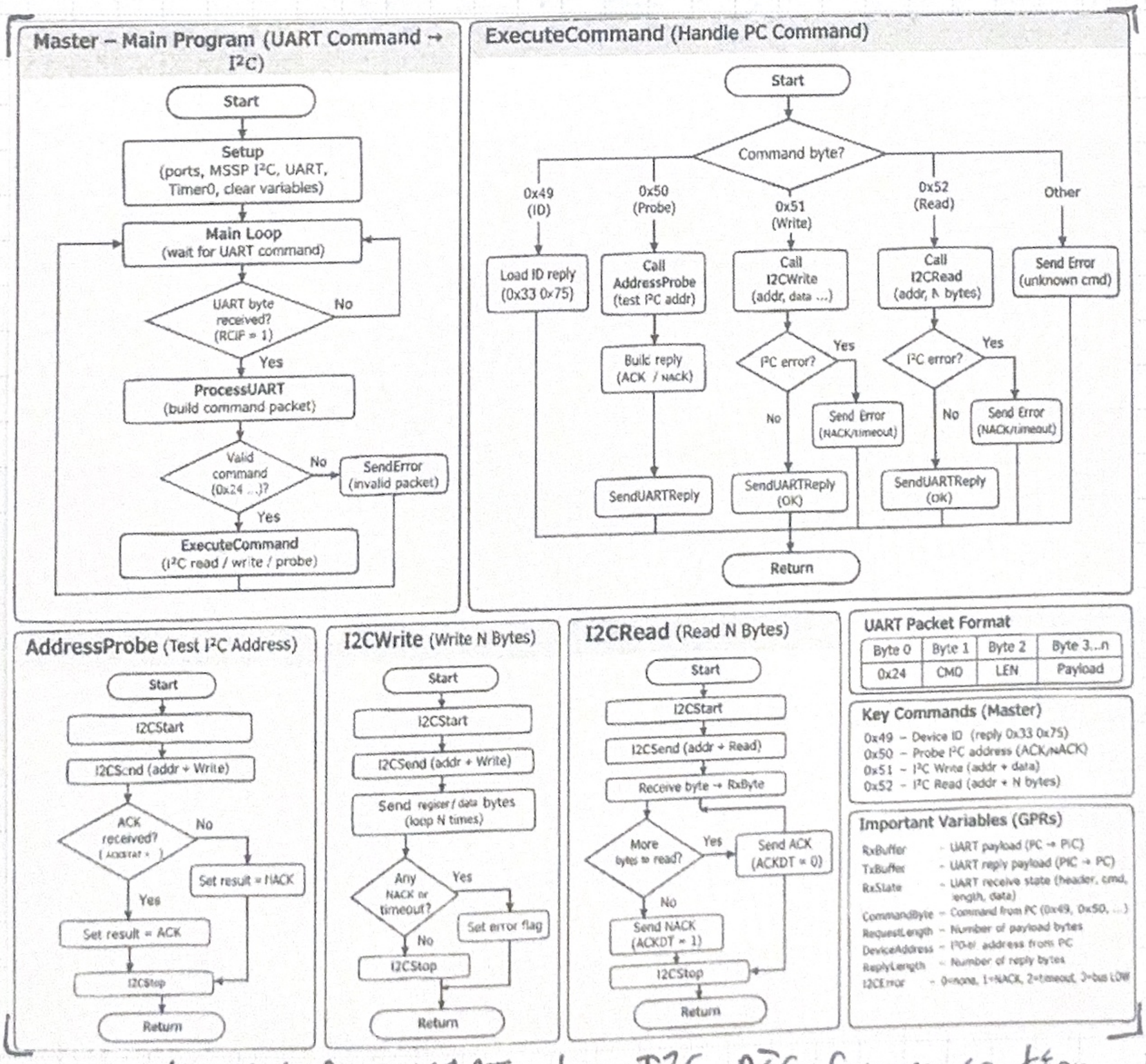
Experiment 10 I²C Communication

ART
11/11/26

Errors & transfer boundaries:

Test	UART TX	UART RX	Expected I²C transaction
Write NACK, absent 21	24 57 02 21 00	24 45 02 04 01	START, 42, NACK, STOP
Read NACK, absent 21	24 52 02 21 01	24 45 02 04 01	START, 43, NACK, STOP
Bad probe length	24 50 00	24 45 02 05 01	None
Unknown command	24 51 00	24 45 02 05 02	None
Invalid address	24 50 01 00	24 45 02 05 03	None
Zero-byte read	24 52 02 20 00	24 45 02 05 01	None
Address-only write	24 57 01 20	24 45 02 05 01	None
Invalid 24-byte read	24 52 02 20 18	24 45 02 05 01	None
Write with 24 data bytes (oversized)	24 57 19 20 01 + 41 × 23	24 45 02 05 01	None
23-byte read (device supports it)	24 52 02 20 17	24 52 17 00 + 00 × 23	START, 41, ACK, (00, ACK) × 22, 00, NACK, STOP
23-byte write (device supports it)	24 57 18 20 01 + 41 × 21 + 42	24 57 01 00	START, 40, ACK, 01, ACK, (41, ACK) × 21, 42, ACK, STOP

Upper-limit examples are conditional: the last two rows show what a successful 23-byte transaction would look like if the selected slave supports that many consecutive bytes. Part3 does not require such streaming behavior. You do not have to modify the slave solely for these tests. Verify the bridge's length validation at the boundary; demonstrate a complete maximum-length transfer if a connected slave supports it. Document the result. An invalid UART request must produce no I²C bus activity.



FC4-1 for UART-to-I²C PIC Communication

ART
4/11/26